```
# ===== frontend/src/components/sandbox/SandBoxMap.vue =====
<template>
 <div class="sandbox-map" :class="{ 'sandbox-map--pseudo': cssFullscreen }">
   <!-- 3D Canvas -->
   <canvas ref="canvasRef" class="sandbox-map__canvas"></canvas>
   <!-- 加载态（构建是异步分片的：转圈动画能真正转起来，并显示进度） -->
   <div v-if="loading" class="sandbox-map__loading">
     <div class="sandbox-map__spinner"></div>
     <span>正在构建 3D 场景…{{ loadingProgress > 0 ? ` ${loadingProgress}%` : '' }}</span>
   </div>
   <!-- 错误态 -->
   <div v-else-if="error" class="sandbox-map__error">
     <span>3D 场景初始化失败</span>
     <button @click="init">重试</button>
   </div>
   <!-- 工具栏 -->
   <div class="sandbox-map__tools">
     <button
       v-for="mode in viewModes"
       :key="mode.key"
       :class="{ 'is-active': viewMode === mode.key }"
       @click="setViewMode(mode.key)"
     >
       {{ mode.label }}
     </button>
     <button class="sandbox-map__reset" @click="resetView" title="回到默认视角">
       <span>⊙</span>
     </button>
     <!-- 全屏查看：浏览器原生 Fullscreen API（Esc 退出由浏览器负责，状态自动同步） -->
     <button
       class="sandbox-map__icon-btn"
       :class="{ 'is-active': isFullscreen }"
       :title="isFullscreen ? '退出全屏' : '全屏查看'"
       :aria-pressed="isFullscreen"
       @click="toggleFullscreen"
     >
       <span>{{ isFullscreen ? '⤡' : '⛶' }}</span>
     </button>
   </div>
   <!-- 罗盘（随相机方位旋转的方位指示，参照 Mapbox NavigationControl 惯例） -->
   <div class="sandbox-map__compass" aria-hidden="true">
     <div class="sandbox-map__compass-ring" :style="{ transform: `rotate(${compassDeg}deg)` }">
       <span class="sandbox-map__compass-n">N</span>
       <i class="sandbox-map__compass-needle sandbox-map__compass-needle--up"></i>
       <i class="sandbox-map__compass-needle sandbox-map__compass-needle--down"></i>
     </div>
   </div>
   <!-- 比例尺（按相机距离实时换算，Mapbox ScaleControl 的同类实现） -->
   <div class="sandbox-map__scale" aria-hidden="true">
     <div class="sandbox-map__scale-bar" :style="{ width: `${scaleBar.px}px` }"></div>
```

```
    <span>{{ scaleBar.label }}</span>
  </div>
  <!-- 缩放拉杆（右侧垂直，对数刻度：顶部=最近） -->
  <div class="sandbox-map__zoom">
    <button class="sandbox-map__zoom-btn" @click="zoomStep(1)" title="放大">+</button>
    <input
      v-model.number="zoomPct"
      type="range"
      min="0"
      max="100"
      step="0.5"
      class="sandbox-map__zoom-slider"
      aria-label="缩放"
      @input="applyZoom"
    />
    <button class="sandbox-map__zoom-btn" @click="zoomStep(-1)" title="缩小">−</button>
  </div>
  <!-- 悬停信息卡 -->
  <div
    v-if="hoverInfo"
    class="sandbox-map__tooltip"
    :style="tooltipStyle"
  >
    <template v-if="hoverInfo.type === 'vessel'">
      <strong>{{ hoverInfo.data.name || hoverInfo.data.robot_id }}</strong>
      <span>状态：{{ statusLabel(hoverInfo.data.status) }}</span>
      <!-- 类型定义：船体=打捞机器人，履带底盘=滩涂清除车（两类都要显示，不只履带） -->
      <span>类型：{{ hoverInfo.data.chassis === 'tracked' ? '滩涂清除车' : '打捞机器人' }}</span>
      <span v-if="hoverInfo.data.battery">电量：{{ hoverInfo.data.battery }}%</span>
      <span class="sandbox-map__tooltip-hint">点击弹出操作</span>
    </template>
    <template v-else-if="hoverInfo.type === 'event'">
      <strong>{{ classLabel(hoverInfo.data.main_class) }}</strong>
      <span>{{ hoverInfo.data.det_count || 1 }} 个目标</span>
      <span class="sandbox-map__tooltip-hint">点击弹出操作</span>
    </template>
    <template v-else-if="hoverInfo.type === 'routeTarget'">
      <strong>作业目标点</strong>
      <span>关联工单：{{ hoverInfo.data.taskId }}</span>
      <span class="sandbox-map__tooltip-hint">点击弹出操作</span>
    </template>
    <template v-else-if="hoverInfo.type === 'debris'">
      <strong>{{ hoverInfo.data.kind === 'beach' ? '沙滩垃圾' : '海浮垃圾' }} · {{ hoverInfo.data…
      <span>点击弹出操作</span>
    </template>
  </div>
  <!-- 选中操作卡：点击物体先选中，确认后才跳转（避免一点就走） -->
  <div
    v-if="selectedInfo"
     class="sandbox-map__action"
```

```
    :style="{
      left: `${selectedPos.x}px`,
      top: `${selectedPos.y + 18}px`,
    }"
  >
    <button
      class="sandbox-map__action-close"
      title="取消选中"
      aria-label="取消选中"
      @click="dismissSelection"
    >×</button>
    <template v-if="selectedInfo.type === 'vessel'">
      <strong>{{ selectedInfo.data.name || selectedInfo.data.robot_id }}</strong>
      <span>状态：{{ statusLabel(selectedInfo.data.status) }}<template v-if="selectedInfo.data.ba…
      <span>类型：{{ selectedInfo.data.chassis === 'tracked' ? '滩涂清除车' : '打捞机器人' }}</sp…
      <span v-if="selectedInfo.data.current_task_id || selectedInfo.data.active_task_id" class="s…
        对应工单：{{ selectedInfo.data.current_task_id || selectedInfo.data.active_task_id }}
      </span>
      <button class="sandbox-map__action-btn" @click="confirmSelection">
        {{ (selectedInfo.data.current_task_id || selectedInfo.data.active_task_id) ? '查看对应工…
      </button>
    </template>
    <template v-else-if="selectedInfo.type === 'event'">
      <strong>{{ classLabel(selectedInfo.data.main_class) }} · 事件点</strong>
      <span>{{ selectedInfo.data.det_count || 1 }} 个识别目标</span>
      <button class="sandbox-map__action-btn" @click="confirmSelection">查看事件详情</button>
    </template>
    <template v-else-if="selectedInfo.type === 'routeTarget'">
      <strong>作业目标点</strong>
      <span>关联工单：{{ selectedInfo.data.taskId }}</span>
      <button class="sandbox-map__action-btn" @click="confirmSelection">进入工单仿真</button>
    </template>
    <template v-else-if="selectedInfo.type === 'debris'">
      <strong>{{ selectedInfo.data.kind === 'beach' ? '沙滩垃圾' : '海浮垃圾' }} · {{ selectedInf…
      <span>按类别筛选事件中心</span>
      <button class="sandbox-map__action-btn" @click="confirmSelection">查看相关事件</button>
    </template>
  </div>
  <!-- 图例 -->
  <div class="sandbox-map__legend">
    <span v-for="item in legendItems" :key="item.key" class="sandbox-map__legend-item">
      <i :style="{ background: item.color }"></i>
      {{ item.label }}
    </span>
  </div>
</div>
</template>
<script setup>
/**
 * 3D 沙盒地图组件
```

```
 *
 * 替代原 Leaflet 2D 地图，提供：
 * - 连江县 3D 地形（程序化生成）
 * - 实时船体位置与状态可视化
 * - 事件点 3D 标记
 * - 点击交互（跳转工单/事件）
 */
import { ref, computed, onMounted, onUnmounted, watch, shallowRef } from 'vue'
import { useRouter } from 'vue-router'
import * as THREE from 'three'
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js'
import { createTerrain, createSeaMaterial, createSkyDome, lngLatToScene, SUN_DIR } from './TerrainL…
import { VesselManager } from './VesselManager.js'
import { InteractionLayer, EventMarkerManager } from './InteractionLayer.js'
import { RouteManager } from './RouteManager.js'
import { DebrisLayer } from './DebrisLayer.js'
import { ReefLayer } from './ReefLayer.js'
import { PostFX } from './PostFX.js'
import { LabelLayer } from './LabelLayer.js'
import { classLabel } from '@/utils/constants'
const props = defineProps({
  robots: { type: Array, default: () => [] },
  events: { type: Array, default: () => [] },
  heatmap: { type: Array, default: () => [] },
  decision: { type: Object, default: null },
  center: { type: Object, default: () => ({ lng: 119.86, lat: 26.35 }) },
  // 航线：[{ robotId, targetLng, targetLat, taskId, phase }]，来自活跃工单
  routes: { type: Array, default: () => [] },
})
const router = useRouter()
// ---------- 状态 ----------
const canvasRef = ref(null)
const loading = ref(true)
const loadingProgress = ref(0) // 地形构建进度 0..100（异步分片构建时回填）
const isFullscreen = ref(false) // 沙盘是否处于（原生或兜底）全屏
const cssFullscreen = ref(false) // 兜底全屏：内嵌 WebView 不兑现 Fullscreen API 时用 CSS 铺满视口
const error = ref('')
const hoverInfo = ref(null)
const tooltipPos = ref({ x: 0, y: 0 })
const zoomPct = ref(20) // 缩放拉杆：0=最远 100=最近（对数刻度）
const compassDeg = ref(0) // 罗盘：北向相对屏幕的旋转角
const scaleBar = ref({ label: '50 km', px: 100 }) // 比例尺
// Three.js 核心对象（shallowRef 避免响应式开销）
const renderer = shallowRef(null)
const scene = shallowRef(null)
const camera = shallowRef(null)
const controls = shallowRef(null)
const vesselManager = shallowRef(null)
const eventManager = shallowRef(null)
const routeManager = shallowRef(null)
```

```
const debrisLayer = shallowRef(null)
const reefLayer = shallowRef(null)
const interaction = shallowRef(null)
const postfx = shallowRef(null)
const labelLayer = shallowRef(null)
let animationId = null
let resizeObserver = null
let clock = null
let trailTimer = 0 // 轨迹记录节流
let seaMaterial = null // 海面材质引用：每帧直接更 uTime，避免 traverse 全场
let seaMesh = null // 海面网格引用：每帧跟随相机 XZ（无限海做法，隐藏平面边缘）
let skyMaterial = null // 天空材质引用：驱动云层漂移
let sceneSpan = 165000 // 场景对角线尺度（米），init 时按地形 bbox 实测更新
let hoverWorldPoint = null // 悬停命中点的世界坐标（tooltip 每帧重投影，相机转动时跟随）
let hudTimer = 0 // HUD（罗盘/比例尺）刷新节流
let cameraFlight = null // 相机飞行动画状态（flyTo）
let followRobotId = null // 跟随模式锁定的船
let followLastPos = null // 上一帧船位（用于差值平移跟随）
let sunLight = null // 主光源引用（阴影取景框动态配置用）
let sunShadowFocus = null // 上次阴影烘焙时的焦点（Vector3）
let sunShadowBox = 0 // 上次阴影取景框边长（米）
let sunShadowBakedAt = 0 // 上次阴影重烤时间（节流用，performance.now）
let rootEl = null // 沙盘根容器（全屏请求对象；unmount 时 ref 已失效，需提前留存）
let userOrbited = false // 用户是否手动转过/缩放过相机（true 时屏幕比例变化不再自动重取景）
let pseudoParent = null // 兜底全屏期间被锁定高度的父容器
let pseudoParentPrevHeight = '' // 其原始内联高度（退出时还原）
// 自适应画质：目标像素比 + FPS 采样窗口
const MAX_PIXEL_RATIO = Math.min(window.devicePixelRatio || 1, 2)
let currentPixelRatio = Math.min(MAX_PIXEL_RATIO, 1.6) // 首帧先给 1.6，弱机开局不崩
let perfAccum = 0
let perfFrames = 0
let lowFpsStreak = 0
const viewMode = ref('perspective')
const viewModes = [
  { key: 'perspective', label: '自由视角' },
  { key: 'top', label: '俯视' },
  { key: 'follow', label: '跟随首船' },
  ]
// ---------- 图例 ----------
const legendItems = computed(() => [
  { key: 'navigating', label: '航行中', color: '#18a999' },
  { key: 'collecting', label: '作业中', color: '#ff9500' },
  { key: 'assigned', label: '已派单', color: '#7a5af8' },
  { key: 'offline', label: '离线', color: '#7a8798' },
  { key: 'event', label: '事件点', color: '#ff3b30' },
  // 垃圾按类别着色（点色与天空光柱同色，与事件标记同一套配色）
  { key: 'debris_foam', label: '泡沫垃圾', color: '#ff9500' },
  { key: 'debris_plastic', label: '塑料垃圾', color: '#007aff' },
  { key: 'debris_fishing', label: '渔具垃圾', color: '#ff3b30' },
  { key: 'debris_other', label: '其他垃圾', color: '#8e8e93' },
```

```
])



// ---------- 展示辅助 ----------
const STATUS_LABELS = {
  online: '在线',
  offline: '离线',
  idle: '待命',
  navigating: '前往中',
  collecting: '作业中',
  assigned: '已派单',
  }
function statusLabel(status) {
  return STATUS_LABELS[status] || status || '未知'
  }
// ----------  tooltip ----------
// tooltip 定位：悬停回调给的是 3D 世界坐标（Vector3），
// 需要用当前相机 project 成屏幕像素，否则提示框会飞到天外。
const tooltipStyle = computed(() => ({
  left: `${tooltipPos.value.x + 14}px`,
  top: `${tooltipPos.value.y + 14}px`,
}))
function worldToScreen(worldPoint) {
  if (!camera.value || !canvasRef.value) return { x: 0, y: 0 }
  const rect = canvasRef.value.getBoundingClientRect()
  const v = worldPoint.clone().project(camera.value)
  return {
    x: rect.left + ((v.x + 1) / 2) * rect.width,
    y: rect.top + ((-v.y + 1) / 2) * rect.height,
  }
  }
// ---------- 初始化 ----------
async function init() {
  if (!canvasRef.value) return
  rootEl = canvasRef.value.parentElement // 全屏对象：整个沙盘容器
  loading.value = true
  loadingProgress.value = 0
  error.value = ''
  try {
    // Renderer
    renderer.value = new THREE.WebGLRenderer({
      canvas: canvasRef.value,
      antialias: true,
      alpha: true,
      powerPreference: 'high-performance',
    })
    renderer.value.setPixelRatio(currentPixelRatio)
    // 山体自阴影：165km 尺度下阴影贴图按"焦点邻域"动态取景（updateSunShadow），
    // 静止场景不重烤阴影（autoUpdate=false + needsUpdate 按需触发）——
    // 阴影是"山体立体感"的最大来源，而按需重烤让它几乎不花帧率
    renderer.value.shadowMap.enabled = true
    // three r186 已移除 PCFSoftShadowMap（会告警并回退），直接用 PCF
```

```
  renderer.value.shadowMap.type = THREE.PCFShadowMap



  renderer.value.shadowMap.autoUpdate = false
  renderer.value.toneMapping = THREE.ACESFilmicToneMapping
  renderer.value.toneMappingExposure = 1.04
  // Scene（雾与海面着色器的距离雾参数一致，远处地形与海天自然融合）
  // 雾起 90km：默认全景视距 ~90km，雾起太近会把整个中景吞成灰白
  scene.value = new THREE.Scene()
  scene.value.fog = new THREE.Fog(0xbcd9ea, 90000, 320000)
  // Camera（场景跨度 ~165km：near/far 拉开到百公里级，远平面必须盖住天空穹顶）
  camera.value = new THREE.PerspectiveCamera(50, 1, 100, 900000)
  // Controls
  controls.value = new OrbitControls(camera.value, renderer.value.domElement)
  controls.value.enableDamping = true
  controls.value.dampingFactor = 0.075
  controls.value.rotateSpeed = 0.72
  controls.value.zoomSpeed = 0.8
  controls.value.panSpeed = 0.6
  controls.value.minDistance = 600     // 最近 600m：船体符号放大到 ~840m 后避免镜头穿模
  controls.value.maxDistance = 400000  // 最远 400km：能俯瞰全域
  controls.value.maxPolarAngle = Math.PI / 2 - 0.08
  // 用户手动拖动/滚轮 → 记录"已接管"，屏幕比例变化时不再自动重取景
  controls.value.addEventListener('start', () => {
    userOrbited = true
  })
  // 滚轮/拖拽改变相机距离时，反向同步缩放拉杆
  controls.value.addEventListener('change', syncZoomFromCamera)
  // 后期处理管线（HDR 泛光 + MSAA 合成）：tone mapping 统一由 OutputPass 收尾
  postfx.value = new PostFX(renderer.value, scene.value, camera.value)
  postfx.value.setPixelRatio(currentPixelRatio)
  // 灯光
  setupLighting()
  // 地形（2048² 网格：~80m/格；重噪声跑在半分辨率、Catmull-Rom 上采样到全分辨率——
  // 山脊/海岸轮廓精度再翻倍。异步分片构建：加载动画可动、页面不冻结）
  const terrain = await createTerrain(
    { grid: 2048, maxHeight: 1400, coastTaper: 16 },
    (p) => {
      loadingProgress.value = Math.round(p * 100)
    },
  )
  if (!canvasRef.value) return // 构建期间组件已卸载：终止后续初始化
  scene.value.add(terrain.group)
  sceneSpan = Math.max(terrain.widthM, terrain.depthM)
  // 海面（注入海岸距离场：近岸浅水色 + 岸线浪花带 + 高度场山影）
  // 网格分段 512：波形靠逐像素解析计算，网格只负责几何位移的平滑度
  seaMaterial = createSeaMaterial({
    coastTex: terrain.coastTex,
    coastBounds: terrain.coastBounds,
    heightTex: terrain.heightTex,
    heightBounds: terrain.heightBounds,
  })
```

```
  const seaGeo = new THREE.PlaneGeometry(terrain.seaSize, terrain.seaSize, 512, 512)


  seaGeo.rotateX(-Math.PI / 2)
  const sea = new THREE.Mesh(seaGeo, seaMaterial)
  sea.receiveShadow = false
  // 海面初始与地形中心对齐；之后每帧跟随相机 XZ（波形锚定世界坐标，不会漂移）
  sea.position.set(terrain.group.children[0].position.x, 0, terrain.group.children[0].position.z)
  scene.value.add(sea)
  seaMesh = sea
  // 天空（fbm 云层，uTime 每帧驱动漂移）
  const skyDome = createSkyDome()
  skyMaterial = skyDome.material
  scene.value.add(skyDome)
  // 相机就位（依赖 sceneSpan 实测值）
  resetCamera()
  syncZoomFromCamera()
  // 首帧前先烤一次阴影（autoUpdate=false，不触发就永远没有影子）
  updateSunShadow()
  // 船体管理器（注入地形高度采样：坐标落在岸上的船会被抬到地形之上）
  vesselManager.value = new VesselManager(scene.value, { sampleHeight: terrain.sampleHeight })
  // 事件标记管理器
  eventManager.value = new EventMarkerManager(scene.value)
  // 航线管理器（已走轨迹 + 剩余航线 + 目标浮标）
  routeManager.value = new RouteManager(scene.value)
  // 垃圾图层（海浮垃圾漂移 + 沙滩垃圾，发光点）
  debrisLayer.value = new DebrisLayer(scene.value, {
    sampleHeight: terrain.sampleHeight,
    bounds: terrain.coastBounds,
  })
  debrisLayer.value.build()
  // 沿岸礁石群（浅水礁石 + 露出水面带浪花环）
  reefLayer.value = new ReefLayer(scene.value, {
    sampleHeight: terrain.sampleHeight,
    bounds: terrain.coastBounds,
  })
  reefLayer.value.build()
  // 沙滩履带机器人已改为"数据驱动的正式机器人"（chassis=tracked，
  // 由 VesselManager 统一渲染/拾取/跳转），不再用程序化巡滩图层
  // 交互层
  interaction.value = new InteractionLayer(camera.value, scene.value, {
    onVesselClick: handleVesselClick,
    onEventClick: handleEventClick,
    onRouteTargetClick: handleRouteTargetClick,
    onDebrisClick: handleDebrisClick,
    onEmptyClick: dismissSelection,
    onHover: handleHover,
  })
  interaction.value.attach(renderer.value.domElement)
  // CSS2D 标签层（船名/事件类别，DOM 覆盖层，不影响 WebGL 分辨率）
  labelLayer.value = new LabelLayer(canvasRef.value.parentElement)
  // 初始数据同步
```

```
    syncData()


    // 启动动画循环（THREE.Timer：接入 Page Visibility，切后台回来不会吃到大 dt）
    clock = new THREE.Timer()
    clock.connect(document)
    animate()
    // 响应式
    setupResize()
    loading.value = false
  } catch (err) {
    console.error('[3D 地图] 初始化失败:', err)
    error.value = err.message || 'WebGL 初始化失败'
    loading.value = false
  }
}
function setupLighting() {
  // 环境光（压低一点：环境光越高，主光的明暗对比越弱、山体越"平"）
  const ambient = new THREE.HemisphereLight(0xd6ecf8, 0x10222b, 0.8)
  scene.value.add(ambient)
  // 主光源（太阳）：方向取自 SUN_DIR（与海面高光/天空日盘同源），
  // 开阴影：山体自阴影是立体感的最大来源；阴影相机取景框由
  // updateSunShadow() 每帧按相机焦点动态配置（165km 场景没法用固定框）
  const sun = new THREE.DirectionalLight(0xfff4e2, 1.8)
  sun.castShadow = true
  sun.shadow.mapSize.set(4096, 4096)
  sun.shadow.camera.near = 10000
  sun.shadow.camera.far = 500000
  sun.shadow.bias = -0.0002
  scene.value.add(sun)
  scene.value.add(sun.target)
  sunLight = sun
  // 补光
  const fill = new THREE.DirectionalLight(0x9fd8ea, 0.4)
  fill.position.set(-90000, 60000, -70000)
  scene.value.add(fill)
}
/**
 * 阴影相机动态取景：以相机焦点为中心、按视距缩放取景框。
 * 全场景 165km 若用固定阴影框，贴图精度会糊成噪点；
 * 反过来说：视距 94km 时框 ~75km（18m/texel），贴近到 4km 时框 ~3km（0.7m/texel）。
 * 场景静止（地形不变、太阳不变）→ autoUpdate=false，只在取景框移动/缩放时重烤。
 */
function updateSunShadow() {
  if (!sunLight || !controls.value || !camera.value || !renderer.value) return
  const focus = controls.value.target
  const dist = camera.value.position.distanceTo(focus)
  // 取景框必须罩住整个可视区（垂直视场 50° 时地面可视跨度 ≈ 0.93×dist）——
  // 之前用 0.8×dist 比可视区小，画面远端会出现"阴影突然消失"的直边接缝
  const box = THREE.MathUtils.clamp(dist * 1.15, 4000, 320000)
  const moved = sunShadowFocus ? focus.distanceTo(sunShadowFocus) : Infinity
  const boxChanged = Math.abs(box - sunShadowBox) > sunShadowBox * 0.06
```

```
  if (!boxChanged && moved < box * 0.04) return


  // 重烤节流：800 万三角面的阴影 pass 不便宜：相机连续移动（切视角/拖拽）时
  // 最多 ~4 次/秒，停止移动后的下一帧会自动补一次最终烘焙
  const now = performance.now()
  if (now - sunShadowBakedAt < 240) return
  sunShadowBakedAt = now
  if (sunShadowFocus) sunShadowFocus.copy(focus)
  else sunShadowFocus = focus.clone()
  sunShadowBox = box
  const offset = box * 1.9 + 40000 // 光源距离：保证阴影相机远近平面的信噪比
  sunLight.position.copy(focus).addScaledVector(SUN_DIR, offset)
  sunLight.target.position.copy(focus)
  sunLight.target.updateMatrixWorld()
  const cam = sunLight.shadow.camera
  cam.left = -box / 2
  cam.right = box / 2
  cam.top = box / 2
  cam.bottom = -box / 2
  cam.near = Math.max(1000, offset - box * 1.6)
  cam.far = offset + box * 1.6
  cam.updateProjectionMatrix()
  // 阴影法线偏移随手放大：尺度越大越要避免自遮挡摩尔纹（acne）
  sunLight.shadow.normalBias = Math.max(4, box * 0.0016)
  renderer.value.shadowMap.needsUpdate = true
  }
// ---------- 相机控制 ----------
// 所有默认距离都按 sceneSpan（场景跨度，~165km）的比例计算，
// 硬编码的公里级距离在这个尺度下会把相机怼进地形里。
//
// ---------- 屏幕适配 ----------
// 取景距离还要按"当前容器的宽高比"换算：168km 宽的群岛在 16:9 上刚好填满，
// 但在窄屏/竖屏（或全屏后比例变化）下会被裁掉两侧：相机按屏比等比拉远，
// 同一片地形在任何屏幕上都能完整入画。
const REF_ASPECT = 16 / 9 // 取景参数调校时的基准屏比
/** 当前容器宽高比（拿不到容器时回退相机 aspect） */
function viewAspect() {
  const el = canvasRef.value?.parentElement
  if (el && el.clientHeight > 0) return el.clientWidth / el.clientHeight
  return camera.value?.aspect || REF_ASPECT
  }
/** 窄屏系数：屏越窄/越竖，相机需等比拉远（宽屏保持 1） */
function narrowBoost() {
  return Math.max(1, REF_ASPECT / viewAspect())
  }
/** 各视角模式的取景位姿（切视角与屏幕比例变化共用同一套算法） */
function framingFor(mode) {
  const { x, z } = lngLatToScene(props.center.lng, props.center.lat)
  const b = narrowBoost()
  if (mode === 'top') {
    // 正俯视：高度略大于跨度，保证最宽边完整入画；
```

```
    // z 给 1m 微偏避免极轴奇异（OrbitControls 在极点是退化姿态）
    return {
      pos: new THREE.Vector3(x, sceneSpan * 1.05 * b, z + 1),
      target: new THREE.Vector3(x, 0, z),
    }
  }
  // 自由视角：视距 ≈ 0.57 倍跨度（~94km）、仰角 ~33°（按屏比等比缩放）
  return {
    pos: new THREE.Vector3(x + sceneSpan * 0.44 * b, sceneSpan * 0.36 * b, z + sceneSpan * 0.44 * b…
    target: new THREE.Vector3(x, 0, z),
  }
  }
/** 屏幕比例变化时"瞬间"按当前模式重新取景（不用飞行动画，拉伸窗口才跟手） */
function refitForScreen() {
  if (!camera.value || !controls.value || viewMode.value === 'follow') return
  const { pos, target } = framingFor(viewMode.value)
  camera.value.position.copy(pos)
  controls.value.target.copy(target)
  camera.value.lookAt(target)
  controls.value.update()
  syncZoomFromCamera()
  updateSunShadow() // 焦点变了：阴影取景框同步（内部有节流）
  }
function resetCamera() {
  const { pos, target } = framingFor('perspective')
  camera.value.position.copy(pos)
  camera.value.lookAt(target)
  controls.value?.target.copy(target)
  }
/**
 * 相机平滑飞行（参考 Google Earth / Mapbox flyTo 的观感）：
 * 位置与视点同时做 easeInOutCubic 插值：视角切换不再是"瞬移"，
 * 这是大屏演示时专业感的第一步。飞行期间禁用控件，避免用户输入打架。
 */
function flyTo(pos, target, duration = 1.3) {
  if (!camera.value || !controls.value) return
  cameraFlight = {
    fromPos: camera.value.position.clone(),
    toPos: pos.clone(),
    fromTarget: controls.value.target.clone(),
    toTarget: target.clone(),
    t: 0,
    dur: Math.max(0.001, duration),
  }
  controls.value.enabled = false
  }
function cancelFlight() {
  if (cameraFlight) {
    cameraFlight = null
    if (controls.value) controls.value.enabled = true
```

```
}



}
function setViewMode(mode) {
  viewMode.value = mode
  userOrbited = false // 重新取景 → 之后屏幕比例再变化仍可自动适配
  followRobotId = null
  followLastPos = null
  if (mode === 'top') {
    const f = framingFor('top')
    flyTo(f.pos, f.target)
  } else if (mode === 'follow') {
    // 跟随第一艘船：低空视角（~4.4km 视距、仰角 ~26°）——
    // 船体、尾流与海面波光入画，又不会低到只剩一片陆坡
    const firstVessel = props.robots[0]
    if (firstVessel) {
      const pos = lngLatToScene(firstVessel.lng || props.center.lng, firstVessel.lat || props.cente…
      followRobotId = firstVessel.robot_id || null
      followLastPos = null
      flyTo(
        new THREE.Vector3(pos.x + 2600, 1800, pos.z + 2600),
        new THREE.Vector3(pos.x, 0, pos.z),
      )
    } else {
      viewMode.value = 'perspective'
      const f = framingFor('perspective')
      flyTo(f.pos, f.target)
    }
  } else {
    const f = framingFor('perspective')
    flyTo(f.pos, f.target)
  }
  }
function resetView() {
  setViewMode('perspective')
  }
// ---------- 全屏 ----------
// 优先用浏览器原生 Fullscreen API 把整个沙盘容器（含 HUD/标签/工具栏）放大到全屏；
// 尺寸变化不用手动处理：容器的 ResizeObserver 会照常触发，渲染器/后期/标签
// 全部跟着重算；Esc 退出由浏览器负责，这里只监听 fullscreenchange 同步按钮状态。
//
// 兜底：内嵌 WebView / 无 allowfullscreen 的 iframe 里 requestFullscreen 会
//   既不 resolve 也不 reject（挂起）——约定时间没等到事件就退化为 CSS 铺满模式，
//   保证按钮在任何环境都不是"点了没反应"。
let fsRequestTimer = null
function fsElement() {
  return document.fullscreenElement || document.webkitFullscreenElement || null
  }
function enableCssFullscreen() {
  clearTimeout(fsRequestTimer)
  cssFullscreen.value = true
```

```
  isFullscreen.value = true



  // 锁定父容器占位高度：fixed 定位会让地图脱离文档流，大屏栅格会因
  //   "地图列变空"重新分配行高：不锁的话退出后布局回不到原样，
  //   容器宽高比漂移还会连带影响相机取景
  const parent = rootEl?.parentElement
  if (parent) {
    pseudoParent = parent
    pseudoParentPrevHeight = parent.style.height
    parent.style.height = `${parent.getBoundingClientRect().height}px`
  }
  // 必须同时抬升 .layout__main 的层叠层级（见文件末尾的全局样式注释）：
  //   覆盖层自身 z-index 再高，也被 main 的层叠上下文困住，越不过顶栏 header
  document.body.classList.add('sandbox-pseudo-fs')
  }
/** 还原兜底全屏期间锁定的父容器高度 */
function releasePseudoPlaceholder() {
  if (pseudoParent) {
    pseudoParent.style.height = pseudoParentPrevHeight
    pseudoParent = null
    pseudoParentPrevHeight = ''
  }
  }
/**
 * 退出全屏后布局可能要一两帧才重排完（栅格/弹性布局回弹），
 * 稳定后再按"恢复后的容器比例"重取景一次：用户未接管时构图一定与屏幕匹配
 */
function scheduleRefitOnSettle() {
  requestAnimationFrame(() => {
    requestAnimationFrame(() => {
      if (!userOrbited && !cssFullscreen.value && !fsElement()) refitForScreen()
    })
  })
  }
function exitFullscreen() {
  clearTimeout(fsRequestTimer)
  if (cssFullscreen.value) {
    cssFullscreen.value = false
    isFullscreen.value = false
    document.body.classList.remove('sandbox-pseudo-fs')
    releasePseudoPlaceholder()
    scheduleRefitOnSettle()
    return
  }
  const exit = document.exitFullscreen || document.webkitExitFullscreen
  if (exit) exit.call(document)
  // 原生退出的状态同步由 fullscreenchange 兜底，这里先乐观置位
  if (!fsElement()) {
    isFullscreen.value = false
    scheduleRefitOnSettle()
  }
```

```
}



function toggleFullscreen() {
  if (isFullscreen.value) {
    exitFullscreen()
    return
  }
  if (!rootEl) return
  const req = rootEl.requestFullscreen || rootEl.webkitRequestFullscreen
  if (!req) {
    enableCssFullscreen()
    return
  }
  try {
    const ret = req.call(rootEl)
    if (ret && typeof ret.then === 'function') {
      ret.catch(() => enableCssFullscreen())
    }
    fsRequestTimer = setTimeout(() => {
      if (!fsElement()) enableCssFullscreen()
    }, 800)
  } catch {
    enableCssFullscreen()
  }
}
function onFullscreenChange() {
  const native = !!fsElement()
  if (native) {
    cssFullscreen.value = false // 原生全屏接管，撤掉兜底层
    isFullscreen.value = true
    document.body.classList.remove('sandbox-pseudo-fs')
    releasePseudoPlaceholder()
  } else if (!cssFullscreen.value) {
    isFullscreen.value = false
    scheduleRefitOnSettle() // 原生全屏退出：等布局稳定后按容器比例重取景
  }
}
/** 兜底模式下浏览器不会处理 Esc，自己监听 */
function onFullscreenKeydown(e) {
  if (e.key === 'Escape' && cssFullscreen.value) exitFullscreen()
}
// ---------- 缩放拉杆 ----------
// 对数刻度：600m ~ 400km 跨度近 3 个数量级，线性滑杆会全挤在一头
function zoomToDist(z) {
  const min = controls.value?.minDistance ?? 600
  const max = controls.value?.maxDistance ?? 400000
  return min * Math.pow(max / min, (100 - z) / 100)
}
function distToZoom(d) {
  const min = controls.value?.minDistance ?? 600
  const max = controls.value?.maxDistance ?? 400000
```

```
  return 100 - (Math.log(d / min) / Math.log(max / min)) * 100



 }
/** 拉杆 → 相机：保持当前视线方向，沿轴向 target 推拉 */
function applyZoom() {
 if (!camera.value || !controls.value) return
 userOrbited = true // 用户主动调了缩放：屏幕比例变化时不再自动覆盖他的构图
 cancelFlight() // 用户接管缩放时立即结束飞行，输入优先
 const dist = zoomToDist(zoomPct.value)
 const dir = camera.value.position.clone().sub(controls.value.target).normalize()
 camera.value.position.copy(controls.value.target).addScaledVector(dir, dist)
 }
/** 相机 → 拉杆：滚轮/拖拽/切视角后同步滑杆位置 */
function syncZoomFromCamera() {
 if (!camera.value || !controls.value) return
 zoomPct.value = distToZoom(camera.value.position.distanceTo(controls.value.target))
 }
function zoomStep(dir) {
 zoomPct.value = Math.max(0, Math.min(100, zoomPct.value + dir * 8))
 applyZoom()
 }
// ---------- 数据同步 ----------
function syncData() {
 if (!vesselManager.value) return
 vesselManager.value.syncVessels(props.robots, props.decision)
 eventManager.value?.syncEvents(props.events)
 routeManager.value?.syncRoutes(props.routes)
 // CSS2D 标签同步（挂点取自各管理器的 group，随实体移动）
 labelLayer.value?.syncVessels(props.robots, vesselManager.value)
 labelLayer.value?.syncEvents(props.events, eventManager.value)
 // 更新交互对象列表
 const interactive = [
   ...vesselManager.value.getInteractiveObjects(),
   ...eventManager.value.getInteractiveObjects(),
   ...routeManager.value?.getInteractiveObjects() || [],
   ...debrisLayer.value?.getInteractiveObjects() || [],
 ]
 interaction.value?.setInteractiveObjects(interactive)
 }
// ---------- 交互处理 ----------
// 两段式交互：点击物体只"选中"并弹出操作卡，用户再点卡上的按钮才真正跳转——
// 沙盘物体在俯视图里彼此很近，一点就走（尤其垃圾点/事件点）太容易误触
const selectedInfo = ref(null) // { type, data } | null
const selectedPos = ref({ x: 0, y: 0 }) // 操作卡屏幕位置（每帧重投影）
let selectedWorldPoint = null
function selectTarget(type, data, point) {
 selectedInfo.value = { type, data }
 selectedWorldPoint = point?.clone ? point.clone() : null
 if (selectedWorldPoint) selectedPos.value = worldToScreen(selectedWorldPoint)
 hoverInfo.value = null // 选中后收起悬停卡，避免两张卡叠在一起
 hoverWorldPoint = null
```

```
}



function dismissSelection() {
  selectedInfo.value = null
  selectedWorldPoint = null
  }
/** 操作卡上的确认按钮：到这里才真正跳转 */
function confirmSelection() {
  const sel = selectedInfo.value
  if (!sel) return
  const d = sel.data
  if (sel.type === 'vessel') {
    const taskId = d.current_task_id || d.active_task_id
    // 有对应工单 → 去工单看板并高亮那一张（抽屉会自动展开）：
    // 直接扎进仿真页会让用户失去"这是我点的那张工单"的上下文
    if (taskId) router.push({ path: '/tasks', query: { task: taskId } })
    else router.push({ path: '/tasks', query: { robot: d.robot_id } })
  } else if (sel.type === 'event') {
    router.push({ name: 'events', query: { event: d.event_id } })
  } else if (sel.type === 'routeTarget') {
    if (d.taskId) router.push({ name: 'simulation', params: { taskId: d.taskId } })
  } else if (sel.type === 'debris') {
    if (d.main_class) router.push({ path: '/events', query: { class: d.main_class } })
  }
  dismissSelection()
  }
function handleVesselClick(robotData, point) {
  selectTarget('vessel', robotData, point)
  }
function handleEventClick(eventData, point) {
  selectTarget('event', eventData, point)
  }
function handleRouteTargetClick(payload, point) {
  selectTarget('routeTarget', payload, point)
  }
/** 点击沙盘上的垃圾点 → 弹出操作卡，确认后去事件中心按类别过滤 */
function handleDebrisClick(info, point) {
  if (!info?.main_class) return
  selectTarget('debris', info, point)
  }
function handleHover(hit) {
  hoverInfo.value = hit
  if (hit?.point) {
    // 存世界坐标，tooltip 位置在渲染循环里每帧重投影：相机转动/缩放时提示框贴着目标走
    hoverWorldPoint = hit.point.clone()
    tooltipPos.value = worldToScreen(hoverWorldPoint)
  } else {
    hoverWorldPoint = null
  }
  }
// ---------- 动画循环 ----------
```

```
801function animate() {
802  animationId = requestAnimationFrame(animate)
803  // dt 上限 0.1s：切后台回来 getDelta 可能是好几秒，直接喂给缓动/飞行会瞬移
804  clock.update()
805  const dt = Math.min(clock.getDelta(), 0.1)
806  const time = clock.getElapsed()
807  // ---- 相机：飞行动画 / 跟随 / 控件更新 ----
808  if (cameraFlight) {
809    cameraFlight.t = Math.min(1, cameraFlight.t + dt / cameraFlight.dur)
810    const k = cameraFlight.t
811    // easeInOutCubic：起步轻、中段快、收尾稳
812    const e = k < 0.5 ? 4 * k * k * k : 1 - Math.pow(-2 * k + 2, 3) / 2
813    camera.value.position.lerpVectors(cameraFlight.fromPos, cameraFlight.toPos, e)
814    controls.value.target.lerpVectors(cameraFlight.fromTarget, cameraFlight.toTarget, e)
815    camera.value.lookAt(controls.value.target)
816    syncZoomFromCamera()
817    if (k >= 1) {
818      cameraFlight = null
819      controls.value.enabled = true
820      controls.value.update()
821    }
822  } else {
823    controls.value?.update()
824  }
825  // 更新船体
826  vesselManager.value?.update(dt, time, camera.value, renderer.value?.domElement?.clientHeight || 8…
827  const positions = vesselManager.value?.getPositions() || null
828  // 跟随模式：按船位差值平移相机与视点：保留用户的旋转/缩放调整，只跟位移
829  if (!cameraFlight && viewMode.value === 'follow' && followRobotId && positions) {
830    const pos = positions.get(followRobotId)
831    if (pos) {
832      if (followLastPos) {
833        const dx = pos.x - followLastPos.x
834        const dz = pos.z - followLastPos.z
835        if (dx !== 0 || dz !== 0) {
836          camera.value.position.x += dx
837          camera.value.position.z += dz
838          controls.value.target.x += dx
839          controls.value.target.z += dz
840        }
841      }
842      followLastPos = { x: pos.x, z: pos.z }
843    }
844  }
845  // 更新事件标记
846  eventManager.value?.update(time)
847  // 更新垃圾图层（海浮垃圾漂移 + 随浪起伏；像素比随自适应画质变化）
848  debrisLayer.value?.update(dt, time, renderer.value?.getPixelRatio() || 1)
849  // 礁石浪花环呼吸
850  reefLayer.value?.update(time)
```

```
// 更新航线（记录轨迹 + 更新 ribbon/圆点/目标浮标）



if (routeManager.value && positions) {
  // 轨迹记录节流：每 0.4 秒检查一次，避免每帧都 push
  trailTimer += dt
  if (trailTimer >= 0.4) {
    trailTimer = 0
    for (const [robotId, pos] of positions) {
      routeManager.value.recordTrailPoint(robotId, pos)
    }
  }
  routeManager.value.update(positions, time)
}
// 更新海面：uTime 驱动波浪 + 网格跟随相机 XZ
//  （波场在着色器里锚定世界坐标，所以挪网格不会让浪"跟着走"）
if (seaMaterial) {
  seaMaterial.uniforms.uTime.value = time
}
if (seaMesh && camera.value) {
  seaMesh.position.x = camera.value.position.x
  seaMesh.position.z = camera.value.position.z
}
if (skyMaterial) {
  skyMaterial.uniforms.uTime.value = time
}
// 交互悬停检测（pointermove 只记坐标，真正的射线检测在这里每帧最多一次）
interaction.value?.update()
// tooltip 每帧重投影：相机转动时提示框跟随目标点
if (hoverInfo.value && hoverWorldPoint) {
  tooltipPos.value = worldToScreen(hoverWorldPoint)
}
// 选中操作卡跟着物体走（相机转动/缩放时重新投影）
if (selectedInfo.value && selectedWorldPoint) {
  selectedPos.value = worldToScreen(selectedWorldPoint)
}
// CSS2D 标签（可见性分级 + DOM 定位）
labelLayer.value?.update(camera.value, dt)
labelLayer.value?.render(scene.value, camera.value)
// 阴影取景框跟随相机焦点（内部有节流：不移动就不重烤）
updateSunShadow()
// HUD（罗盘/比例尺）节流刷新
hudTimer += dt
if (hudTimer >= 0.25) {
  hudTimer = 0
  updateHUD()
}
// 自适应画质（FPS 采样窗口）
monitorPerformance(dt)
// 渲染（走后期管线：HDR 泛光 + OutputPass 色调映射收尾）
if (postfx.value) {
  postfx.value.render()
```

```
  } else {
    renderer.value?.render(scene.value, camera.value)
  }
 }
// ---------- HUD ----------
function updateHUD() {
  if (!camera.value || !controls.value || !canvasRef.value) return
  // 罗盘：北向（-z）相对屏幕上方偏转角的负值：镜头朝东时 N 指向屏幕左侧
  const dir = controls.value.target.clone().sub(camera.value.position)
  compassDeg.value = -THREE.MathUtils.radToDeg(Math.atan2(dir.x, -dir.z))
  // 比例尺：按视点平面上的"每像素世界距离"换算，取整到易读档位
  const height = canvasRef.value.clientHeight || 600
  const dist = camera.value.position.distanceTo(controls.value.target)
  const worldPerPx = (2 * dist * Math.tan(THREE.MathUtils.degToRad(camera.value.fov / 2))) / height
  const targetPx = Math.min(140, Math.max(80, (canvasRef.value.clientWidth || 800) * 0.16))
  const steps = [200, 500, 1000, 2000, 5000, 10000, 20000, 50000, 100000, 200000, 500000]
  let picked = steps[0]
  for (const s of steps) {
    if (s <= worldPerPx * targetPx) picked = s
  }
  scaleBar.value = {
    label: picked >= 1000 ? `${picked / 1000} km` : `${picked} m`,
    px: Math.round(picked / worldPerPx),
  }
 }
// ---------- 自适应画质 ----------
/**
 * FPS 窗口监控（2s 一次判定）：
 *   低帧 → 逐档降像素比（1.6 → 1.0），仍低则关泛光；充裕 → 逐档升回。
 * 大屏演示机千差万别，交给运行时自己找平衡，比赌一个固定值稳。
 */
function monitorPerformance(dt) {
  perfAccum += dt
  perfFrames += 1
  if (perfAccum < 2) return
  const fps = perfFrames / perfAccum
  perfAccum = 0
  perfFrames = 0
  if (fps < 42) {
    lowFpsStreak += 1
    if (currentPixelRatio > 1) {
      currentPixelRatio = Math.max(1, currentPixelRatio - 0.3)
      applyPixelRatio()
    } else if (lowFpsStreak >= 2 && postfx.value && postfx.value.bloom.enabled) {
      postfx.value.bloom.enabled = false // 保帧率优先：场景本体清晰比泛光重要
    }
  } else if (fps > 57) {
    lowFpsStreak = 0
    if (postfx.value && !postfx.value.bloom.enabled) {
      postfx.value.bloom.enabled = true
```

```
    } else if (currentPixelRatio < MAX_PIXEL_RATIO) {
      currentPixelRatio = Math.min(MAX_PIXEL_RATIO, currentPixelRatio + 0.25)
      applyPixelRatio()
    }
  }
  }
function applyPixelRatio() {
  renderer.value?.setPixelRatio(currentPixelRatio)
  postfx.value?.setPixelRatio(currentPixelRatio)
  }
// ---------- 响应式 ----------
function setupResize() {
  const parent = canvasRef.value?.parentElement
  if (!parent) return
  let lastAspect = 0
  const resize = () => {
    const width = parent.clientWidth
    const height = parent.clientHeight
    if (!width || !height) return
    const aspect = width / height
    camera.value.aspect = aspect
    camera.value.updateProjectionMatrix()
    renderer.value.setSize(width, height, false)
    postfx.value?.setSize(width, height)
    labelLayer.value?.setSize(width, height)
    // 屏幕比例变化（窗口拉伸/旋转屏/进出全屏）：用户还没手动转过相机时，
    //   按当前视角模式重新取景：默认构图在任何屏幕比例下都完整入画
    if (lastAspect && Math.abs(aspect - lastAspect) / lastAspect > 0.06 && !userOrbited) {
      refitForScreen()
    }
    lastAspect = aspect
  }
  resize()
  resizeObserver = new ResizeObserver(resize)
  resizeObserver.observe(parent)
  }
// ---------- 生命周期 ----------
onMounted(() => {
  init()
  // 全屏状态同步（含 Esc / F11 等浏览器侧退出）+ 兜底模式的 Esc 处理
  document.addEventListener('fullscreenchange', onFullscreenChange)
  document.addEventListener('webkitfullscreenchange', onFullscreenChange)
  document.addEventListener('keydown', onFullscreenKeydown)
 })
onUnmounted(() => {
  if (animationId) cancelAnimationFrame(animationId)
  clearTimeout(fsRequestTimer)
  document.body.classList.remove('sandbox-pseudo-fs')
  releasePseudoPlaceholder()
   document.removeEventListener('fullscreenchange', onFullscreenChange)
```

```
  document.removeEventListener('webkitfullscreenchange', onFullscreenChange)


  document.removeEventListener('keydown', onFullscreenKeydown)
  // 卸载时若本组件仍是全屏元素，主动退出：否则会留下一个没有内容的黑屏全屏
  if (rootEl && fsElement() === rootEl) {
    const exit = document.exitFullscreen || document.webkitExitFullscreen
    if (exit) exit.call(document)
  }
  clock?.disconnect()
  resizeObserver?.disconnect()
  interaction.value?.dispose()
  labelLayer.value?.dispose()
  vesselManager.value?.dispose()
  eventManager.value?.dispose()
  debrisLayer.value?.dispose()
  reefLayer.value?.dispose()
  routeManager.value?.dispose()
  postfx.value?.dispose()
  renderer.value?.dispose()
})
// ---------- 监听数据变化 ----------
watch(
  () => [props.robots, props.events, props.decision, props.routes],
  () => syncData(),
  { deep: true }
)
</script>
<style scoped>
.sandbox-map {
  position: relative;
  width: 100%;
  height: 100%;
  /* 高度下限随视口自适应：小屏笔记本不被地图撑破，大屏给足画面 */
  min-height: clamp(320px, 44vh, 560px);
  overflow: hidden;
  border-radius: inherit;
  background: linear-gradient(145deg, #0a1f2a, #061a26);
}
.sandbox-map__canvas {
  width: 100%;
  height: 100%;
  display: block;
}
.sandbox-map__loading {
  position: absolute;
  inset: 0;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 12px;
```

```
  background: rgba(6, 26, 38, 0.9);
}
.sandbox-map__spinner {
  width: 32px;
  height: 32px;
  border: 3px solid rgba(255, 255, 255, 0.1);
  border-top-color: #18a999;
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
}
@keyframes spin {
  to { transform: rotate(360deg); }
}
.sandbox-map__error {
  position: absolute;
  inset: 0;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 12px;
  background: rgba(6, 26, 38, 0.9);
  color: #ff3b30;
  font-size: 14px;
  z-index: 10;
}
.sandbox-map__error button {
  padding: 8px 16px;
  border: 1px solid #ff3b30;
  border-radius: 8px;
  background: transparent;
  color: #ff3b30;
  cursor: pointer;
}
.sandbox-map__tools {
  position: absolute;
  top: 12px;
  right: 12px;
  display: flex;
  gap: 6px;
  z-index: 5;
}
.sandbox-map__tools button {
  padding: 6px 12px;
  border: 1px solid rgba(255, 255, 255, 0.2);
  border-radius: 8px;
  background: rgba(0, 0, 0, 0.4);
```

```
  color: rgba(255, 255, 255, 0.8);
  font-size: 12px;
  cursor: pointer;
  backdrop-filter: blur(8px);
  transition: all 0.2s;
}
.sandbox-map__tools button:hover,
.sandbox-map__tools button.is-active {
  background: rgba(24, 169, 153, 0.3);
  border-color: #18a999;
  color: #18a999;
}
.sandbox-map__reset,
.sandbox-map__icon-btn {
  width: 36px;
  padding: 0 !important;
  font-size: 16px !important;
}
/* ---------- 全屏 ---------- */
/* 全屏元素由浏览器拉伸到屏幕：显式补上尺寸与去圆角，
   Safari 旧版只认 -webkit- 前缀伪类 */
.sandbox-map:fullscreen,
.sandbox-map:-webkit-full-screen {
  width: 100vw;
  height: 100vh;
  border-radius: 0;
}
/* 兜底全屏（内嵌 WebView 不兑现 Fullscreen API 时）：
   CSS 铺满视口并盖过顶栏(300)/抽屉(900)，行为与真全屏一致 */
.sandbox-map--pseudo {
  position: fixed;
  inset: 0;
  z-index: 1200;
  width: 100vw;
  height: 100vh;
  border-radius: 0;
}
/* ---------- 罗盘 ---------- */
.sandbox-map__compass {
  position: absolute;
  top: 12px;
  left: 12px;
  width: 54px;
  height: 54px;
  border-radius: 50%;
  background: rgba(0, 0, 0, 0.4);
  border: 1px solid rgba(255, 255, 255, 0.14);
  backdrop-filter: blur(8px);
  z-index: 5;
}
```

```
.sandbox-map__compass-ring {
  position: relative;
  width: 100%;
  height: 100%;
  transition: transform 0.18s linear;
}
.sandbox-map__compass-n {
  position: absolute;
  top: 3px;
  left: 50%;
  transform: translateX(-50%);
  font-size: 10px;
  font-weight: 600;
  color: #ff6b5e;
  line-height: 1;
}
.sandbox-map__compass-needle {
  position: absolute;
  left: 50%;
  width: 0;
  height: 0;
  border-left: 4px solid transparent;
  border-right: 4px solid transparent;
  transform: translateX(-50%);
}
.sandbox-map__compass-needle--up {
  top: 15px;
  border-bottom: 11px solid #ff5d52;
}
.sandbox-map__compass-needle--down {
  bottom: 15px;
  border-top: 11px solid rgba(255, 255, 255, 0.85);
}
/* ---------- 比例尺 ---------- */
.sandbox-map__scale {
  position: absolute;
  right: 12px;
  bottom: 12px;
  display: flex;
  flex-direction: column;
  align-items: flex-end;
  gap: 4px;
  z-index: 5;
  color: rgba(255, 255, 255, 0.88);
  font-size: 11px;
  text-shadow: 0 1px 3px rgba(0, 0, 0, 0.6);
  pointer-events: none;
}
.sandbox-map__scale-bar {
  height: 7px;
```

```
  min-width: 24px;
  border: 1px solid rgba(255, 255, 255, 0.8);
  border-top: none;
  background: linear-gradient(
    90deg,
    rgba(255, 255, 255, 0.6) 0 50%,
    rgba(255, 255, 255, 0.1) 50% 100%
  );
}
/* ---------- 缩放拉杆 ---------- */
.sandbox-map__zoom {
  position: absolute;
  right: 12px;
  top: 50%;
  transform: translateY(-50%);
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 8px;
  padding: 10px 8px;
  background: rgba(0, 0, 0, 0.4);
  border: 1px solid rgba(255, 255, 255, 0.12);
  border-radius: 12px;
  backdrop-filter: blur(8px);
  z-index: 5;
}
.sandbox-map__zoom-btn {
  width: 26px;
  height: 26px;
  border: 1px solid rgba(255, 255, 255, 0.2);
  border-radius: 8px;
  background: rgba(0, 0, 0, 0.4);
  color: rgba(255, 255, 255, 0.85);
  font-size: 15px;
  line-height: 1;
  cursor: pointer;
  transition: all 0.2s;
}
.sandbox-map__zoom-btn:hover {
  background: rgba(24, 169, 153, 0.3);
  border-color: #18a999;
  color: #18a999;
}
/* 垂直滑杆：Chrome/Edge/Safari 走 -webkit-appearance,
   Firefox 走标准 writing-mode；两个方案都是 max 在顶端 */
.sandbox-map__zoom-slider {
  -webkit-appearance: slider-vertical;
  appearance: slider-vertical;
  writing-mode: vertical-lr;
  direction: rtl;
```

```
  width: 8px;
 }
 .sandbox-map__tooltip {
  position: fixed;
  padding: 10px 14px;
  background: rgba(10, 31, 42, 0.95);
  border: 1px solid rgba(255, 255, 255, 0.15);
  border-radius: 10px;
  color: #fff;
  font-size: 12px;
  pointer-events: none;
  z-index: 100;
  display: flex;
  flex-direction: column;
  gap: 4px;
  backdrop-filter: blur(12px);
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.4);
 }
 .sandbox-map__tooltip strong {
  font-size: 13px;
 }
 .sandbox-map__tooltip-hint {
  color: #18a999;
  font-size: 11px;
  margin-top: 4px;
 }
 /* ---------- 选中操作卡（两段式交互的第二段） ---------- */
 .sandbox-map__action {
  position: fixed;
  transform: translateX(-50%);
  min-width: 190px;
  max-width: 240px;
  padding: 12px 14px 12px;
  background: rgba(10, 31, 42, 0.97);
  border: 1px solid rgba(24, 169, 153, 0.45);
  border-radius: 12px;
  color: #fff;
  font-size: 12px;
  z-index: 120;
  display: flex;
  flex-direction: column;
  gap: 5px;
  backdrop-filter: blur(12px);
  box-shadow: 0 10px 28px rgba(0, 0, 0, 0.5);
  animation: sandbox-map-action-in 0.16s ease-out;
```

```
}


@keyframes sandbox-map-action-in {
  from {
    opacity: 0;
    transform: translateX(-50%) translateY(-6px) scale(0.96);
  }
  to {
    opacity: 1;
    transform: translateX(-50%) translateY(0) scale(1);
  }
}
.sandbox-map__action strong {
  font-size: 13px;
  padding-right: 16px;
}
.sandbox-map__action span {
  color: rgba(255, 255, 255, 0.72);
}
.sandbox-map__action-task {
  color: #18a999 !important;
  font-size: 11px;
}
.sandbox-map__action-close {
  position: absolute;
  top: 6px;
  right: 8px;
  width: 18px;
  height: 18px;
  padding: 0;
  border: none;
  border-radius: 50%;
  background: transparent;
  color: rgba(255, 255, 255, 0.55);
  font-size: 14px;
  line-height: 1;
  cursor: pointer;
}
.sandbox-map__action-close:hover {
  background: rgba(255, 255, 255, 0.12);
  color: #fff;
}
.sandbox-map__action-btn {
  margin-top: 6px;
  padding: 7px 10px;
  border: none;
  border-radius: 8px;
  background: #18a999;
  color: #041e1a;
  font-size: 12px;
  font-weight: 600;
```

```
 cursor: pointer;
 transition: filter 0.15s ease;
}
.sandbox-map__action-btn:hover {
 filter: brightness(1.12);
}
.sandbox-map__action-btn:active {
 filter: brightness(0.94);
}
.sandbox-map__legend {
 position: absolute;
 bottom: 12px;
 left: 12px;
 display: flex;
 flex-wrap: wrap;
 gap: 8px 16px;
 padding: 8px 12px;
 background: rgba(0, 0, 0, 0.4);
 border-radius: 10px;
 backdrop-filter: blur(8px);
 z-index: 5;
}
.sandbox-map__legend-item {
 display: flex;
 align-items: center;
 gap: 6px;
 font-size: 11px;
 color: rgba(255, 255, 255, 0.8);
}
.sandbox-map__legend-item i {
 width: 10px;
 height: 10px;
 border-radius: 50%;
 display: inline-block;
}
</style>
<!-- 标签层样式必须非 scoped：DOM 由 CSS2DRenderer 动态创建，拿不到 scoped 属性 -->
<style>
/* 兜底全屏时把主内容层抬到顶栏之上：
   App.vue 里 .layout__main 的 z-index:1 会形成层叠上下文，
   覆盖层自身的 z-index 再高也越不过兄弟节点 header 的 300——
   不抬这一层，全屏后的工具栏/退出按钮会被顶栏盖住（点不到）。 */
body.sandbox-pseudo-fs .layout__main {
 z-index: 400;
}
.sandbox-map__labels {
 position: absolute;
 inset: 0;
 overflow: hidden;
 pointer-events: none;
```

```
  z-index: 4;



}
.sb-label {
  display: flex;
  align-items: center;
  gap: 5px;
  padding: 3px 8px;
  border-radius: 999px;
  font-size: 11px;
  line-height: 1.35;
  font-family: inherit;
  color: #eaf6ff;
  background: rgba(6, 26, 38, 0.74);
  border: 1px solid rgba(255, 255, 255, 0.14);
  backdrop-filter: blur(6px);
  white-space: nowrap;
  pointer-events: none;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.35);
  transition: opacity 0.2s;
}
.sb-label i {
  width: 7px;
  height: 7px;
  border-radius: 50%;
  flex: none;
}
.sb-label em {
  font-style: normal;
  color: #8fd8cd;
}
.sb-label--vessel.is-selected {
  border-color: rgba(255, 149, 0, 0.85);
  box-shadow: 0 0 12px rgba(255, 149, 0, 0.35);
}
.sb-label--vessel.is-candidate {
  border-color: rgba(122, 90, 248, 0.75);
}
.sb-label--event {
  font-size: 10px;
  padding: 2px 7px;
  background: rgba(6, 26, 38, 0.66);
}
.sb-label--event.is-new {
  animation: sb-label-pop 0.9s ease;
}
.sb-label.is-hidden {
  display: none;
}
/* 注意：不能动画 transform：CSS2DRenderer 用 inline transform 定位元素，
   动画 transform 会覆盖定位导致标签飞走。用透明度 + 光晕表达"新事件" */
```

```
@keyframes sb-label-pop {
  0% { opacity: 0; }
  55% { opacity: 1; box-shadow: 0 0 16px rgba(255, 255, 255, 0.5); }
  100% { opacity: 1; box-shadow: 0 2px 8px rgba(0, 0, 0, 0.35); }
}
</style>
# ===== frontend/src/components/sandbox/TerrainLayer.js =====
/**
 * 连江 3D 地形生成器（网格位移版）
 *
 * 为什么不续用 ExtrudeGeometry：
 *   挤出几何体的顶面顶点只落在多边形轮廓上，内部没有网格点，做不了山地起伏；
 *   且连江 GeoJSON 有数千个轮廓点，earcut 三角化慢、三角形数量不可控。
 *
 * 本版方案（一张 PlaneGeometry 解决全部）：
 *   1. 256×256 网格覆盖整个连江 bbox（含远海岛屿），逐顶点做射线法包含判定
 *   2. 陆地顶点：基础高度 × 海岸衰减（腐蚀窗口）× fbm 噪声起伏 → 自然山丘
 *   3. 海域顶点：压到海平面以下，海面单独用波浪着色器覆盖
 *   4. 顶点色按海拔分层：沙滩 → 植被 → 山体，无需贴图也有层次
 *   5. 轮廓环先过 Douglas-Peucker 简化，包含判定提速一个量级
 */
import * as THREE from 'three'
import lianjiangGeo from '@/assets/lianjiang-350122.json'
// 连江中心点
const CENTER_LNG = lianjiangGeo.features[0]?.properties?.center?.[0] || 119.86
const CENTER_LAT = lianjiangGeo.features[0]?.properties?.center?.[1] || 26.35
const METERS_PER_DEG_LAT = 110540
const METERS_PER_DEG_LNG = 111320 * Math.cos((CENTER_LAT * Math.PI) / 180)
/**
 * 海天接缝色：海面距离雾的终点色 == 天空穹顶"地平线以下"的底色。
 * 两处必须同源：有限大的海面在远处收进这个颜色，若天空地平线以下是另一个
 * 浅色，海面边缘（世界尽头）就会显出一条硬边。
 */
const SEA_FOG_COLOR = 0xa9c9dd
/**
 * 全场景统一太阳方向（单位向量）。
 * 海面高光 / 天空日盘 / 地形凹凸阴影必须同源：否则"波光在左、影子在右"，
 * 立体感会互相打架。
 *
 * 方位取西南、仰角 ~35°（下午的太阳，符合东海岸真实光照）：
 * 关键是与默认相机方位"垂直"——默认镜头从东南上方看向西北，
 *   若太阳也在东南（顺光），山体只有受光面、投影全藏在山背后，立体感为零；
 *   垂直侧光下每道山脊都有明暗两面，体积感才真正出来（地形渲染的行业惯例）。
 */
export const SUN_DIR = new THREE.Vector3(-0.55, 0.55, 0.55).normalize()
/** 经纬度转场景坐标（以连江中心为原点，单位≈米） */
export function lngLatToScene(lng, lat) {
  const x = (lng - CENTER_LNG) * METERS_PER_DEG_LNG
  const z = -(lat - CENTER_LAT) * METERS_PER_DEG_LAT
  return { x, z }
```

```
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
  .sim-head__action {
    width: 100%;
  }
  .sim-layout {
    grid-template-columns: minmax(0, 1fr);
    flex: none;
  }
  .sim-map {
    height: clamp(380px, 58vh, 560px);
  }
  .sim-side {
    max-height: none;
  }
  .sim-side__scroll {
    overflow: visible;
  }
  .sim-stream__tabs {
    position: static;
  }
  .sim-controls {
    position: sticky;
    bottom: 0;
    z-index: 20;
    flex-wrap: wrap;
    gap: 8px;
    padding-bottom: calc(9px + env(safe-area-inset-bottom, 0px));
  }
  .sim-controls__primary {
    flex: 1 1 100%;
  }
  .sim-controls__primary .sim-control {
    flex: 1;
  }
}
@media (max-width: 560px) {
  .sim-head__title h1 {
    font-size: 14px;
  }
  .sim-state {
    margin-left: auto;
  }
  .sim-layout {
    gap: 10px;
  }
  .sim-map {
    height: 420px;
  }
  .sim-controls__speeds {
```

```
  flex: 1;
}
.sim-controls__speeds button {
  flex: 1;
  min-width: 0;
}
.sim-controls__secondary {
  flex: 1;
}
.sim-controls__secondary .sim-control {
  flex: 1;
}
.sim-stream__row {
  grid-template-columns: 54px 38px minmax(0, 1fr);
  gap: 5px;
}
}
</style>
# ===== frontend/src/components/MapPanelPrecision.vue =====
<template>
<div class="map-panel panel">
  <div class="panel-title">
    <span>垃圾分布热力与设备态势</span>
    <div class="map-legend">
      <span v-for="c in legend" :key="c.key" class="map-legend__item">
        <i class="dot" :style="{ background: c.color }"></i>{{ c.label }}
      </span>
    </div>
  </div>
  <!-- 本次派单决策：把"选了谁、为什么"直接写在图上方，不用去翻轨迹 -->
  <div v-if="decision && decision.selected_robot_id" class="map-decision">
    <span class="map-decision__tag">本次派单决策</span>
    <span class="map-decision__text">
      {{ decision.selected_robot_name || decision.selected_robot_id }}
      <template v-if="decision.distance_m != null"> ({{ Math.round(Number(decision.distance_m)) }…
      — {{ decision.reason || '按策略择优' }}
    </span>
    <span v-if="decision.candidates?.length" class="map-decision__count">
      候选 {{ decision.candidates.length }}
    </span>
  </div>
  <div class="map-panel__body">
    <div
      id="seasight-map"
      ref="mapEl"
      :class="{ 'seasight-map--hidden': mapMode === 'offline' }"
    ></div>
    <svg
      v-if="mapMode === 'offline'"
      class="offline-map"
```

```
      viewBox="0 0 1000 700"



      role="img"
      aria-label="连江县海区离线态势图"
    >
      <defs>
        <linearGradient id="precision-water" x1="0" y1="0" x2="1" y2="1">
          <stop offset="0" stop-color="#dcecf5" />
          <stop offset="1" stop-color="#c9deeb" />
        </linearGradient>
        <linearGradient id="precision-land" x1="0" y1="0" x2="1" y2="1">
          <stop offset="0" stop-color="#fbfcfa" />
          <stop offset="1" stop-color="#edf1ed" />
        </linearGradient>
        <filter id="precision-shadow" x="-20%" y="-20%" width="140%" height="140%">
          <feDropShadow dx="0" dy="5" stdDeviation="6" flood-color="#526b7d" flood-opacity="0.18"…
        </filter>
      </defs>
      <rect width="1000" height="700" fill="url(#precision-water)" />
      <path
        d="M-30 112C120 58 258 84 374 146S650 252 1030 172"
        fill="none"
        stroke="#ffffff"
        stroke-opacity="0.22"
        stroke-width="1.2"
      />
      <path
        d="M-34 196C116 142 250 170 370 228S664 332 1038 246"
        fill="none"
        stroke="#9fc5dd"
        stroke-opacity="0.16"
      />
      <path class="offline-map__coast" :d="lianjiangOutline" />
      <path
        class="offline-map__land"
        :d="lianjiangOutline"
        filter="url(#precision-shadow)"
      />
      <g v-if="layerEnabled('heat')" class="offline-map__heat">
        <circle
          v-for="cell in localHeat"
          :key="cell.key"
          :cx="cell.x"
          :cy="cell.y"
          :r="cell.radius"
          :fill="cell.color"
          :fill-opacity="cell.opacity"
        />
      </g>
      <g v-if="layerEnabled('cameras')">
        <circle
```

```
          v-for="marker in localCameras"
          :key="marker.id"
          :cx="marker.x"
          :cy="marker.y"
          r="5.5"
          :fill="marker.color"
          stroke="#ffffff"
          stroke-width="1.6"
        />
      </g>
      <g v-if="layerEnabled('robots')">
        <circle
          v-for="marker in localRobots"
          :key="marker.id"
          :cx="marker.x"
          :cy="marker.y"
          r="7"
          :fill="marker.color"
          stroke="#ffffff"
          stroke-width="1.6"
        />
      </g>
      <g v-if="layerEnabled('events')">
        <circle
          v-for="marker in localEvents"
          :key="marker.id"
          :cx="marker.x"
          :cy="marker.y"
          r="4.5"
          :fill="marker.color"
          stroke="#ffffff"
          stroke-width="1.5"
        />
      </g>
      <text class="offline-map__water-label" x="700" y="560">黄岐湾</text>
      <text class="offline-map__ocean-label" x="865" y="625">东 海</text>
    </svg>
    <div v-if="mapMode !== 'loading'" class="map-tools">
      <div class="map-tools__base" role="group" aria-label="底图类型">
        <button
          type="button"
          :class="{ 'is-active': baseMode === 'satellite' }"
          @click="setBaseMode('satellite')"
        >
          高清
        </button>
        <button
          type="button"
          :class="{ 'is-active': baseMode === 'earth' }"
          @click="setBaseMode('earth')"
```

```
1701          >
1702            影像
1703          </button>
1704          <button
1705            type="button"
1706            :class="{ 'is-active': baseMode === 'street' }"
1707            @click="setBaseMode('street')"
1708          >
1709            街道
1710          </button>
1711        </div>
1712        <label v-for="layer in layers" :key="layer.key" class="map-tools__item">
1713          <input type="checkbox" v-model="layer.on" @change="refreshOverlays()" />
1714          <span>{{ layer.label }}</span>
1715        </label>
1716      </div>
1717      <div v-if="mapMode === 'offline'" class="map-controls" aria-label="离线地图缩放控制">
1718        <button type="button" title="放大" aria-label="放大地图" @click="changeOfflineZoom(0.2)">+<…
1719        <button type="button" title="缩小" aria-label="缩小地图" @click="changeOfflineZoom(-0.2)">-…
1720        <button type="button" title="复位" aria-label="复位地图" @click="resetOfflineView">⊙</butto…
1721      </div>
1722      <div v-if="mapMode === 'precision'" class="map-source-badge">
1723        <span class="map-source-badge__dot"></span>
1724        <span>{{ sourceBadge.title }}</span>
1725        <small>{{ sourceBadge.subtitle }}</small>
1726      </div>
1727      <div v-if="mapMode === 'offline'" class="map-source-badge map-source-badge--offline" :title="…
1728        <span class="map-source-badge__dot"></span>
1729        <span>连江县本地海区态势图</span>
1730        <small>{{ offlineReason }}</small>
1731      </div>
1732      <div v-if="selectedMarker" class="offline-info" role="dialog" aria-label="点位详情">
1733        <button type="button" aria-label="关闭点位详情" @click="selectedMarker = null">×</button>
1734        <strong>{{ selectedMarker.title }}</strong>
1735        <span v-for="row in selectedMarker.rows" :key="row.label">
1736          {{ row.label }}: {{ row.value }}
1737        </span>
1738      </div>
1739      <div v-if="mapMode === 'loading'" class="map-loading">
1740        <span class="map-loading__spinner" aria-hidden="true"></span>
1741        <span>正在加载高清影像</span>
1742      </div>
1743    </div>
1744  </div>
1745</template>
1746<script setup>
1747import { computed, onMounted, onUnmounted, reactive, ref, shallowRef, watch } from 'vue'
1748import L from 'leaflet'
1749import 'leaflet/dist/leaflet.css'
1750import { classColor, classLabel, CLASS_ORDER } from '@/utils/constants'
```

```
import lianjiangGeo from '@/assets/lianjiang-350122.json'



const props = defineProps({
  heatmap: { type: Array, default: () => [] },
  devices: { type: Array, default: () => [] },
  robots: { type: Array, default: () => [] },
  events: { type: Array, default: () => [] },
  center: { type: Object, default: () => ({ lng: 119.86, lat: 26.35 }) },
  zoom: { type: Number, default: 11 },
  // 最近一次派单决策快照（AgentRunOut.decision）：用于给机器人 marker 上色、
  // 在气泡里补理由、并在图上方写一句"选了谁、为什么"。为 null 时地图回到
  // 纯态势展示，不显示任何决策痕迹。
  decision: { type: Object, default: null },
})
const AMAP_SATELLITE_URL =
  'https://webst0{s}.is.autonavi.com/appmaptile?style=6&x={x}&y={y}&z={z}'
const AMAP_STREET_URL =
  'https://webrd0{s}.is.autonavi.com/appmaptile?lang=zh_cn&size=1&scale=1&style=7&x={x}&y={y}&z={z}'
const ESRI_EARTH_URL =
  'https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}'
const ESRI_LABELS_URL =
  'https://server.arcgisonline.com/ArcGIS/rest/services/Reference/World_Boundaries_and_Places/MapSe…
const GCJ_AXIS = 6378245
const GCJ_EE = 0.00669342162296594323
const PI = Math.PI
const mapEl = ref(null)
const map = shallowRef(null)
const mapMode = ref('loading')
const baseMode = ref('satellite')
const offlineReason = ref('高清影像暂不可用')
const selectedMarker = ref(null)
const localZoom = ref(1)
const localPan = reactive({ x: 0, y: 0 })
let satelliteLayer = null
let earthLayer = null
let earthLabelsLayer = null
let streetLayer = null
let overlayLayer = null
// 覆盖物登记表：`${类型}:${业务 id}` → Leaflet layer。
// WS 高频推送下不再 clearLayers 全量重建（重建会打断开着的 popup、marker 闪烁），
// refreshOverlays 对照新数据做增/改/删。
const overlayMarkers = new Map()
let resizeObserver = null
let tileWatchdog = null
let tileLoadCount = 0
let tileErrorCount = 0
const layers = reactive([
  { key: 'heat', label: '热力分布', on: true },
  { key: 'cameras', label: '摄像头', on: true },
  { key: 'robots', label: '机器人', on: true },
  { key: 'events', label: '事件点', on: true },
```

```
])



const legend = computed(() => {
  const items = CLASS_ORDER.map((key) => ({
    key,
    label: classLabel(key),
    color: classColor(key),
  }))
  // 有派单决策叠加时，把决策色也写进图例：图上出现了新颜色的点，
  // 图例不说明就成了"看不懂的彩点"。
  const decisionRoles = new Set((props.robots || []).map((r) => r.decision_role).filter(Boolean))
  for (const [role, tone] of Object.entries(DECISION_STYLE)) {
    if (decisionRoles.has(role)) {
      items.push({ key: `decision-${role}`, label: tone.label, color: tone.color })
    }
  }
  return items
})
const usesGcj02 = computed(() => baseMode.value !== 'earth')
const sourceBadge = computed(() => {
  if (baseMode.value === 'earth') {
    return {
      title: 'Esri 全球卫星影像',
      subtitle: 'World Imagery · 19 级原生细节',
    }
  }
  if (baseMode.value === 'street') {
    return {
      title: '高德中文街道底图',
      subtitle: '道路与行政区注记 · 18 级原生细节',
    }
  }
  return {
    title: '高德高清卫星影像',
    subtitle: '高清卫星底图 · 18 级原生细节',
  }
})
const MAP_WIDTH = 1000
const MAP_HEIGHT = 700
const MAP_PADDING = 44
function collectCoordinates(geometry, result = []) {
  if (!geometry) return result
  if (geometry.type === 'Polygon') {
    geometry.coordinates.flat().forEach((coordinate) => result.push(coordinate))
  } else if (geometry.type === 'MultiPolygon') {
    geometry.coordinates.flat(2).forEach((coordinate) => result.push(coordinate))
  }
  return result
}
const geoCoordinates = lianjiangGeo.features.flatMap((feature) =>
  collectCoordinates(feature.geometry),
```

```
)


const GEO_BOUNDS = {
  minLng: Math.min(...geoCoordinates.map(([lng]) => lng)),
  maxLng: Math.max(...geoCoordinates.map(([lng]) => lng)),
  minLat: Math.min(...geoCoordinates.map(([, lat]) => lat)),
  maxLat: Math.max(...geoCoordinates.map(([, lat]) => lat)),
}
function projectLngLat(lng, lat) {
  const width = MAP_WIDTH - MAP_PADDING * 2
  const height = MAP_HEIGHT - MAP_PADDING * 2
  return {
    x: MAP_PADDING + ((lng - GEO_BOUNDS.minLng) / (GEO_BOUNDS.maxLng - GEO_BOUNDS.minLng)) * width,
    y: MAP_PADDING + ((GEO_BOUNDS.maxLat - lat) / (GEO_BOUNDS.maxLat - GEO_BOUNDS.minLat)) * height,
  }
}
function ringToPath(ring) {
  return `${ring
    .map(([lng, lat], index) => {
      const point = projectLngLat(lng, lat)
      return `${index ? 'L' : 'M'}${point.x.toFixed(2)} ${point.y.toFixed(2)}`
    })
    .join(' ')} Z`
}
function geometryToPath(geometry) {
  if (!geometry) return ''
  if (geometry.type === 'Polygon') {
    return geometry.coordinates.map(ringToPath).join(' ')
  }
  if (geometry.type === 'MultiPolygon') {
    return geometry.coordinates.flatMap((polygon) => polygon.map(ringToPath)).join(' ')
  }
  return ''
}
const lianjiangOutline = lianjiangGeo.features
  .map((feature) => geometryToPath(feature.geometry))
  .join(' ')
const maxHeatDensity = computed(() =>
  Math.max(...(props.heatmap || []).map((cell) => Number(cell.density) || 0), 1e-9),
)
const localHeat = computed(() =>
  (props.heatmap || []).map((cell, index) => {
    const ratio = Math.min(1, (Number(cell.density) || 0) / maxHeatDensity.value)
    return {
      key: `${cell.lng}-${cell.lat}-${index}`,
      ...projectLngLat(Number(cell.lng), Number(cell.lat)),
      radius: 10 + ratio * 28,
      color: classColor('foam'),
      opacity: 0.12 + ratio * 0.46,
    }
  }),
```

```
)


function markerRows(type, item) {
  if (type === 'camera') {
    return [
      { label: '类型', value: '摄像头' },
      { label: '编号', value: item.device_id },
      { label: '状态', value: item.status === 'online' ? '在线' : '离线' },
    ]
  }
  if (type === 'robot') {
    const rows = [
      { label: '类型', value: '治理机器人' },
      { label: '编号', value: item.robot_id },
      { label: '状态', value: item.status || '未知' },
      { label: '电量', value: item.battery == null ? '未知' : `${item.battery}%` },
    ]
    // 决策联动：气泡里补上"它在这次决策里是什么角色、为什么选它"。
    // 没有决策信息时不加这几行，普通设备气泡保持原样。
    const tone = decisionTone(item)
    if (tone) rows.push({ label: '决策', value: tone.label })
    if (item.decision_reason) rows.push({ label: '理由', value: item.decision_reason })
    if (item.decision_distance_m != null) {
      rows.push({ label: '距事件', value: `${Math.round(Number(item.decision_distance_m))} m` })
    }
    if (item.decision_same_category) rows.push({ label: '顺路', value: '有同类在跑，可顺路' })
    return rows
  }
  return [
    { label: '设备', value: item.device_id || '未关联' },
    { label: '数量', value: item.det_count || 1 },
    {
      label: '置信度',
      value: item.max_confidence
        ? `${(Number(item.max_confidence) * 100).toFixed(0)}%`
        : '未知',
    },
  ]
}
const localCameras = computed(() =>
  (props.devices || [])
    .filter((device) => device.device_type !== 'robot')
    .map((device) => ({
      id: device.device_id,
      ...projectLngLat(Number(device.lng), Number(device.lat)),
      color: device.status === 'online' ? '#18a999' : '#7a8798',
      title: device.name || device.device_id,
      rows: markerRows('camera', device),
    })),
)
const localRobots = computed(() =>
```

```
  (props.robots || []).map((robot) => ({
    id: robot.robot_id,
    ...projectLngLat(Number(robot.lng), Number(robot.lat)),
    // 与在线图层共用同一套配色（含派单决策高亮），离线态不能是另一套口径
    color: robotColor(robot),
    title: robot.name || robot.robot_id,
    rows: markerRows('robot', robot),
  })),
)
const localEvents = computed(() =>
  (props.events || []).map((event, index) => ({
    id: event.event_id || `event-${index}`,
    ...projectLngLat(Number(event.lng), Number(event.lat)),
    color: classColor(event.main_class),
    title: `${classLabel(event.main_class)} ${event.det_count || 1} 个`,
    rows: markerRows('event', event),
  })),
)
const offlineTransform = computed(() => {
  const centerX = MAP_WIDTH / 2
  const centerY = MAP_HEIGHT / 2
  return [
    `translate(${localPan.x} ${localPan.y})`,
    `translate(${centerX} ${centerY})`,
    `scale(${localZoom.value})`,
    `translate(${-centerX} ${-centerY})`,
  ].join(' ')
})
function layerEnabled(key) {
  return layers.find((layer) => layer.key === key)?.on !== false
}
function validLatLng(item) {
  const lng = Number(item.lng)
  const lat = Number(item.lat)
  return Number.isFinite(lng) && Number.isFinite(lat) && lng >= -180 && lng <= 180 && lat >= -90 &&…
}
function outsideChina(lng, lat) {
  return lng < 72.004 || lng > 137.8347 || lat < 0.8293 || lat > 55.8271
}
function transformLat(lng, lat) {
  let ret =
    -100 +
    2 * lng +
    3 * lat +
    0.2 * lat * lat +
    0.1 * lng * lat +
    0.2 * Math.sqrt(Math.abs(lng))
  ret += ((20 * Math.sin(6 * lng * PI) + 20 * Math.sin(2 * lng * PI)) * 2) / 3
  ret += ((20 * Math.sin(lat * PI) + 40 * Math.sin((lat / 3) * PI)) * 2) / 3
  ret +=
```

```
    ((160 * Math.sin((lat / 12) * PI) + 320 * Math.sin((lat * PI) / 30)) * 2) /
    3
  return ret
}
function transformLng(lng, lat) {
  let ret =
    300 +
    lng +
    2 * lat +
    0.1 * lng * lng +
    0.1 * lng * lat +
    0.1 * Math.sqrt(Math.abs(lng))
  ret += ((20 * Math.sin(6 * lng * PI) + 20 * Math.sin(2 * lng * PI)) * 2) / 3
  ret += ((20 * Math.sin(lng * PI) + 40 * Math.sin((lng / 3) * PI)) * 2) / 3
  ret +=
    ((150 * Math.sin((lng / 12) * PI) + 300 * Math.sin((lng / 30) * PI)) * 2) /
    3
  return ret
}
function wgs84ToGcj02(lng, lat) {
  if (outsideChina(lng, lat)) return [lng, lat]
  const dLat = transformLat(lng - 105, lat - 35)
  const dLng = transformLng(lng - 105, lat - 35)
  const radLat = (lat / 180) * PI
  let magic = Math.sin(radLat)
  magic = 1 - GCJ_EE * magic * magic
  const sqrtMagic = Math.sqrt(magic)
  const outLat =
    lat +
    (dLat * 180) /
      (((GCJ_AXIS * (1 - GCJ_EE)) / (magic * sqrtMagic)) * PI)
  const outLng =
    lng +
    (dLng * 180) /
      ((GCJ_AXIS / sqrtMagic) * Math.cos(radLat) * PI)
  return [outLng, outLat]
}
function displayLatLng(item) {
  const lng = Number(item.lng)
  const lat = Number(item.lat)
  const [shownLng, shownLat] = usesGcj02.value
    ? wgs84ToGcj02(lng, lat)
    : [lng, lat]
  return [shownLat, shownLng]
}
function escapeHtml(value) {
  return String(value ?? '')
    .replaceAll('&', '&amp;')
    .replaceAll('<', '&lt;')
    .replaceAll('>', '&gt;')
```

```
    .replaceAll('"', '&quot;')
    .replaceAll("'", '&#039;')
}
function popupContent(title, rows) {
  return `
    <div class="seasight-popup">
      <strong>${escapeHtml(title)}</strong>
      ${rows
        .map(
          (row) =>
            `<span><b>${escapeHtml(row.label)}</b>${escapeHtml(row.value)}</span>`,
        )
        .join('')}
    </div>
  `
}
function vectorIcon(type, color, ring = false) {
  // ring：派单决策选中的机器人加一圈虚线光环，静态截图里也能一眼认出来
  //  （只靠颜色的话，投影仪偏色时"橙"和"蓝"会难分）。
  const robotRing = ring
    ? `<circle cx="16" cy="16" r="15.2" fill="none" stroke="${color}" stroke-width="1.5" stroke-das…
    : ''
  const html =
    type === 'robot'
      ? `<svg viewBox="0 0 32 32" aria-hidden="true">
           ${robotRing}
           <circle cx="16" cy="16" r="14" fill="${color}" opacity=".2"/>
           <path d="M7 18h18l-3.5 5H10.5L7 18Z" fill="${color}" stroke="#fff" stroke-width="1.5"/>
           <path d="M10.5 17h11l-2.2-6H12.7l-2.2 6Z" fill="${color}" stroke="#fff" stroke-width="1.…
         </svg>`
      : `<svg viewBox="0 0 32 38" aria-hidden="true">
           <path d="M16 2C9.4 2 4 7.4 4 14c0 8.8 12 22 12 22s12-13.2 12-22C28 7.4 22.6 2 16 2Z"
                 fill="${color}" stroke="#fff" stroke-width="1.6"/>
           <circle cx="16" cy="14" r="4.1" fill="#122330" opacity=".7"/>
         </svg>`
  return L.divIcon({
    className: `seasight-marker seasight-marker--${type}`,
    html,
    iconSize: type === 'robot' ? [34, 34] : [30, 36],
    iconAnchor: type === 'robot' ? [17, 17] : [15, 34],
    popupAnchor: type === 'robot' ? [0, -16] : [0, -31],
  })
}
function createBaseLayers() {
  satelliteLayer = L.tileLayer(AMAP_SATELLITE_URL, {
    subdomains: ['1', '2', '3', '4'],
    minZoom: 3,
    maxNativeZoom: 18,
    maxZoom: 20,
    detectRetina: true,
```

```
    crossOrigin: true,
    attribution: 'Imagery &copy; <a href="https://www.amap.com/">高德地图</a>',
    className: 'seasight-satellite-tiles',
  })
  earthLayer = L.tileLayer(ESRI_EARTH_URL, {
    minZoom: 3,
    maxNativeZoom: 19,
    maxZoom: 20,
    detectRetina: true,
    crossOrigin: true,
    attribution:
      'Tiles &copy; Esri, Maxar, Earthstar Geographics, and the GIS User Community',
    className: 'seasight-satellite-tiles',
  })
  earthLabelsLayer = L.tileLayer(ESRI_LABELS_URL, {
    pane: 'seasight-labels',
    minZoom: 3,
    maxNativeZoom: 19,
    maxZoom: 20,
    detectRetina: true,
    crossOrigin: true,
    opacity: 0.9,
    className: 'seasight-label-tiles',
  })
  streetLayer = L.tileLayer(AMAP_STREET_URL, {
    subdomains: ['1', '2', '3', '4'],
    minZoom: 3,
    maxNativeZoom: 18,
    maxZoom: 20,
    detectRetina: true,
    crossOrigin: true,
    attribution: 'Map &copy; <a href="https://www.amap.com/">高德地图</a>',
    className: 'seasight-street-tiles',
  })
  ;[satelliteLayer, earthLayer, earthLabelsLayer, streetLayer].forEach((layer) => {
    layer.on('tileload', () => {
      tileLoadCount += 1
    })
    layer.on('tileerror', () => {
      tileErrorCount += 1
    })
  })
}
function installBaseLayer() {
  if (!map.value) return
  ;[satelliteLayer, earthLayer, earthLabelsLayer, streetLayer].forEach((layer) => {
    if (layer && map.value.hasLayer(layer)) map.value.removeLayer(layer)
  })
  if (baseMode.value === 'street') {
    if (!streetLayer) createBaseLayers()
```

```
    streetLayer.addTo(map.value)


    return
  }
  if (baseMode.value === 'earth') {
    if (!earthLayer) createBaseLayers()
    earthLayer.addTo(map.value)
    earthLabelsLayer.addTo(map.value)
    return
  }
  if (!satelliteLayer) createBaseLayers()
  satelliteLayer.addTo(map.value)
}
function setBaseMode(mode) {
  if (baseMode.value === mode || !map.value) return
  baseMode.value = mode
  tileLoadCount = 0
  tileErrorCount = 0
  installBaseLayer()
  // 底图换了坐标系（GCJ-02 ↔ WGS84），所有点位必须重算，走全量重建
  refreshOverlays(true)
  if (validLatLng(props.center)) {
    map.value.setView(displayLatLng(props.center), Number(props.zoom) || map.value.getZoom(), {
      animate: true,
    })
  }
}
/**
 * 某一类覆盖物的增量同步：对照 overlayMarkers 做增/改/删。
 * keyOf 给每条数据一个稳定业务 id；create 新建 layer；update 原地改已有 layer。
 */
function syncOverlayKind(kind, items, keyOf, create, update) {
  if (!overlayLayer) return
  const seen = new Set()
  for (const item of items) {
    const key = `${kind}:${keyOf(item)}`
    seen.add(key)
    const existing = overlayMarkers.get(key)
    if (existing) {
      update(existing, item)
    } else {
      const layer = create(item)
      layer.addTo(overlayLayer)
      overlayMarkers.set(key, layer)
    }
  }
  for (const [key, layer] of overlayMarkers) {
    if (key.startsWith(`${kind}:`) && !seen.has(key)) {
      overlayLayer.removeLayer(layer)
      overlayMarkers.delete(key)
    }
```

```
2201  }
2202 }
2203function clearOverlayKind(kind) {
2204  if (!overlayLayer) return
2205  for (const [key, layer] of overlayMarkers) {
2206    if (key.startsWith(`${kind}:`)) {
2207      overlayLayer.removeLayer(layer)
2208      overlayMarkers.delete(key)
2209    }
2210  }
2211 }
2212function robotColor(robot) {
2213  const tone = decisionTone(robot)
2214  if (tone) return tone.color
2215  const busy = ['navigating', 'collecting', 'assigned'].includes(robot.status)
2216  const online = robot.status === 'online' || busy
2217  return busy ? '#18a999' : online ? '#3f8ee8' : '#7a8798'
2218 }
2219// ---------- 派单决策联动 ----------
2220// 大屏上的机器人 marker 本来就在画。给它们多带一个 decision_role 字段，
2221// 就能把"这次派单选了谁、还有哪些候选"直接标在图上：换来的是
2222// 评委不用读表格轨迹，抬眼就能看出决策落点。色系沿用 Agent 控制台：
2223// 选中＝橙、候选＝紫（与研判 Agent 同色系）。
2224const DECISION_STYLE = {
2225  selected: { color: '#ff9500', label: '本次派单选中' },
2226  candidate: { color: '#7a5af8', label: '本次派单候选' },
2227 }
2228function decisionTone(robot) {
2229  return DECISION_STYLE[robot?.decision_role] || null
2230 }
2231function syncHeat() {
2232  const cells = (props.heatmap || []).filter(validLatLng)
2233  const maxDensity = Math.max(...cells.map((cell) => Number(cell.density) || 0), 1e-9)
2234  const styleOf = (cell) => {
2235    const ratio = Math.min(1, (Number(cell.density) || 0) / maxDensity)
2236    return { radius: 180 + ratio * 540, fillOpacity: 0.1 + ratio * 0.38 }
2237  }
2238  syncOverlayKind(
2239    'heat',
2240    cells,
2241    (cell) => `${cell.lng}-${cell.lat}`,
2242    (cell) =>
2243      L.circle(displayLatLng(cell), {
2244        pane: 'seasight-overlays',
2245        stroke: false,
2246        fillColor: classColor('foam'),
2247        interactive: false,
2248        className: 'seasight-leaflet-heat',
2249        ...styleOf(cell),
2250      }),
```

```
    (layer, cell) => {
      const style = styleOf(cell)
      layer.setLatLng(displayLatLng(cell))
      layer.setRadius(style.radius)
      layer.setStyle({ fillOpacity: style.fillOpacity })
    },
  )
}
function cameraPopup(device) {
  return popupContent(device.name || device.device_id, markerRows('camera', device))
}
function cameraIcon(device) {
  return vectorIcon('camera', device.status === 'online' ? '#18a999' : '#7a8798')
}
function syncCameras() {
  syncOverlayKind(
    'camera',
    (props.devices || []).filter((device) => device.device_type !== 'robot' && validLatLng(device)),
    (device) => device.device_id,
    (device) => {
      const marker = L.marker(displayLatLng(device), {
        icon: cameraIcon(device),
        title: device.name || device.device_id,
        riseOnHover: true,
      })
      marker.bindPopup(cameraPopup(device), {
        className: 'seasight-popup-shell',
        closeButton: true,
        offset: [0, 4],
      })
      return marker
    },
    (marker, device) => {
      marker.setLatLng(displayLatLng(device))
      marker.setIcon(cameraIcon(device))
      marker.setPopupContent(cameraPopup(device))
    },
  )
}
function robotPopup(robot) {
  return popupContent(robot.name || robot.robot_id, markerRows('robot', robot))
}
function syncRobots() {
  syncOverlayKind(
    'robot',
    (props.robots || []).filter(validLatLng),
    (robot) => robot.robot_id,
    (robot) => {
      const marker = L.marker(displayLatLng(robot), {
        icon: vectorIcon('robot', robotColor(robot), robot.decision_role === 'selected'),
```

```
      title: robot.name || robot.robot_id,
      riseOnHover: true,
    })
    marker.bindPopup(robotPopup(robot), {
      className: 'seasight-popup-shell',
      closeButton: true,
      offset: [0, 2],
    })
    return marker
  },
  (marker, robot) => {
    marker.setLatLng(displayLatLng(robot))
    marker.setIcon(vectorIcon('robot', robotColor(robot), robot.decision_role === 'selected'))
    marker.setPopupContent(robotPopup(robot))
  },
 )
}
function eventPopup(event) {
 return popupContent(
  `${classLabel(event.main_class)} ${event.det_count || 1} 个`,
  markerRows('event', event),
 )
}
function syncEvents() {
 syncOverlayKind(
  'event',
  (props.events || []).filter(validLatLng),
  // 事件必有 event_id；坐标+类别兜底只是防御
  (event) => event.event_id || `${event.lng}-${event.lat}-${event.main_class}`,
  (event) => {
    const marker = L.circleMarker(displayLatLng(event), {
      pane: 'seasight-overlays',
      radius: 6.5,
      color: '#ffffff',
      weight: 1.8,
      fillColor: classColor(event.main_class),
      fillOpacity: 0.96,
      className: 'seasight-event-marker',
    })
    marker.bindPopup(eventPopup(event), {
      className: 'seasight-popup-shell',
      closeButton: true,
      offset: [0, -2],
    })
    return marker
  },
  (marker, event) => {
    marker.setLatLng(displayLatLng(event))
    marker.setStyle({ fillColor: classColor(event.main_class) })
    marker.setPopupContent(eventPopup(event))
```

```
    },
  )
}
/**
 * rebuild=true 用于底图坐标系切换（GCJ-02 ⇔ WGS84）：所有点位都要重算，
 * 增量更新没有意义，直接清空重建。图层开关变化走增量：关掉的类整类移除。
 */
function refreshOverlays(rebuild = false) {
  if (!map.value) return
  if (!overlayLayer) {
    overlayLayer = L.layerGroup().addTo(map.value)
  }
  if (rebuild) {
    overlayLayer.clearLayers()
    overlayMarkers.clear()
  }
  if (layerEnabled('heat')) syncHeat()
  else clearOverlayKind('heat')
  if (layerEnabled('cameras')) syncCameras()
  else clearOverlayKind('camera')
  if (layerEnabled('robots')) syncRobots()
  else clearOverlayKind('robot')
  if (layerEnabled('events')) syncEvents()
  else clearOverlayKind('event')
}
function initializeOffline(reason) {
  offlineReason.value = reason
  mapMode.value = 'offline'
  if (map.value) {
    map.value.remove()
    map.value = null
  }
}
function initializePrecisionMap() {
  if (!mapEl.value || map.value) return
  mapMode.value = 'loading'
  const instance = L.map(mapEl.value, {
    center: displayLatLng(props.center),
    zoom: Number(props.zoom) || 11,
    minZoom: 3,
    maxZoom: 20,
    zoomControl: false,
    attributionControl: true,
    preferCanvas: true,
    wheelPxPerZoomLevel: 110,
    zoomSnap: 0.5,
    zoomDelta: 1,
  })
  L.control.zoom({ position: 'topleft' }).addTo(instance)
  instance.createPane('seasight-labels')
```

```
  instance.getPane('seasight-labels').style.zIndex = 410
  instance.getPane('seasight-labels').style.pointerEvents = 'none'
  instance.createPane('seasight-overlays')
  instance.getPane('seasight-overlays').style.zIndex = 430
  map.value = instance
  createBaseLayers()
  installBaseLayer()
  refreshOverlays()
  mapMode.value = 'precision'
  requestAnimationFrame(() => instance.invalidateSize())
  tileWatchdog = window.setTimeout(() => {
    if (tileLoadCount === 0 && tileErrorCount > 2) {
      initializeOffline('高清影像瓦片加载失败，已切换本地态势图')
    }
  }, 12000)
}
function clampLocalZoom(value) {
  return Math.min(2.8, Math.max(0.8, value))
}
function changeOfflineZoom(delta) {
  localZoom.value = clampLocalZoom(localZoom.value + delta)
}
function resetOfflineView() {
  localZoom.value = 1
  localPan.x = 0
  localPan.y = 0
  selectedMarker.value = null
}
// 覆盖物签名：只拼"会影响画面"的字段（业务 id + 坐标 + 状态/数值）。
// store 每次 WS 推送都可能整体替换数组引用，盯引用等于每次都全量重画；
// 盯签名则内容没变就不动手，变了也交给增量同步去改。
const overlaySignature = computed(() =>
  [
    (props.heatmap || []).map((c) => `${c.lng},${c.lat},${c.density}`).join('|'),
    (props.devices || []).map((d) => `${d.device_id},${d.lng},${d.lat},${d.status}`).join('|'),
    (props.robots || [])
      .map((r) => `${r.robot_id},${r.lng},${r.lat},${r.status},${r.battery},${r.decision_role || ''…
      .join('|'),
    (props.events || [])
      .map((e) => `${e.event_id},${e.lng},${e.lat},${e.main_class},${e.det_count},${e.max_confidenc…
      .join('|'),
  ].join('#'),
)
watch(overlaySignature, () => refreshOverlays())
watch(
  () => [props.center?.lng, props.center?.lat, props.zoom],
  () => {
    if (!map.value) return
    map.value.setView(
      displayLatLng(props.center),
```

```
      Number(props.zoom) || map.value.getZoom(),
      { animate: true },
    )
  },
)
onMounted(() => {
  initializePrecisionMap()
  if (mapEl.value && window.ResizeObserver) {
    resizeObserver = new ResizeObserver(() => map.value?.invalidateSize())
    resizeObserver.observe(mapEl.value)
  }
})
onUnmounted(() => {
  window.clearTimeout(tileWatchdog)
  resizeObserver?.disconnect()
  overlayLayer = null
  overlayMarkers.clear()
  if (map.value) {
    map.value.remove()
    map.value = null
  }
})
</script>
<style scoped>
.map-panel {
  display: flex;
  flex-direction: column;
  height: 100%;
  min-height: 0;
}
.map-panel__body {
  position: relative;
  flex: 1;
  min-height: 360px;
  overflow: hidden;
  border-radius: 0 0 var(--radius-lg) var(--radius-lg);
  background: #d7e4ec;
}
#seasight-map {
  width: 100%;
  height: 100%;
  min-height: 360px;
  background: #d7e4ec;
}
.seasight-map--hidden {
  visibility: hidden;
}
.offline-map {
  position: absolute;
  inset: 0;
```

```
  z-index: 1;



  width: 100%;
  height: 100%;
  background: #dceaf5;
}
.offline-map__land {
  fill: #f9fbf8;
}
.offline-map__coast {
  fill: #eef3ef;
  stroke: #b7c8d3;
  stroke-width: 1.2;
  stroke-linejoin: round;
  vector-effect: non-scaling-stroke;
}
.offline-map__heat {
  pointer-events: none;
}
.offline-map__water-label,
.offline-map__ocean-label {
  fill: #729fbe;
  stroke: rgba(229, 240, 248, 0.74);
  stroke-width: 2.4;
  paint-order: stroke;
}
.offline-map__water-label {
  font-size: 11px;
  letter-spacing: 2.4px;
}
.offline-map__ocean-label {
  fill: #83a8c1;
  font-size: 15px;
  letter-spacing: 8px;
  opacity: 0.72;
}
.map-controls {
  position: absolute;
  z-index: 1000;
  top: 56px;
  left: 10px;
  display: flex;
  flex-direction: column;
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.78);
  border-radius: 13px;
  background: rgba(255, 255, 255, 0.84);
  box-shadow:
    0 1px 2px rgba(24, 44, 58, 0.08),
    0 8px 24px rgba(24, 44, 58, 0.14);
  backdrop-filter: blur(18px) saturate(1.25);
```

```
  -webkit-backdrop-filter: blur(18px) saturate(1.25);
}
.map-controls button {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 40px;
  height: 40px;
  padding: 0;
  border: 0;
  background: transparent;
  color: #3f4a51;
  font-size: 18px;
  cursor: pointer;
}
.map-controls button + button {
  border-top: 1px solid rgba(60, 60, 67, 0.13);
}
.map-controls button:hover {
  background: rgba(255, 255, 255, 0.9);
  color: #007aff;
}
.map-source-badge {
  position: absolute;
  z-index: 1000;
  bottom: 24px;
  left: 10px;
  display: grid;
  grid-template-columns: auto 1fr;
  align-items: center;
  gap: 0 7px;
  padding: 8px 11px;
  border: 1px solid rgba(255, 255, 255, 0.82);
  border-radius: 11px;
  background: rgba(252, 253, 254, 0.84);
  color: #35434b;
  box-shadow:
    0 1px 2px rgba(37, 51, 61, 0.08),
    0 7px 22px rgba(37, 51, 61, 0.12);
  backdrop-filter: blur(18px) saturate(1.25);
  -webkit-backdrop-filter: blur(18px) saturate(1.25);
  pointer-events: none;
}
.map-source-badge__dot {
  width: 7px;
  height: 7px;
  border-radius: 50%;
  background: #34c759;
  box-shadow: 0 0 0 4px rgba(52, 199, 89, 0.13);
}
```

```
.map-source-badge > span:nth-child(2) {
  font-size: 11px;
  font-weight: 600;
}
.map-source-badge small {
  grid-column: 2;
  color: #71808a;
  font-size: 10px;
  line-height: 1.3;
}
.map-source-badge--offline .map-source-badge__dot {
  background: #ff9500;
  box-shadow: 0 0 0 4px rgba(255, 149, 0, 0.13);
}
.offline-info {
  position: absolute;
  z-index: 1100;
  top: 10px;
  left: 54px;
  display: flex;
  flex-direction: column;
  min-width: 190px;
  max-width: min(280px, calc(100% - 190px));
  gap: 4px;
  padding: 11px 34px 11px 12px;
  border: 1px solid var(--panel-border);
  border-radius: 12px;
  background: color-mix(in srgb, var(--bg-panel) 96%, transparent);
  box-shadow: 0 12px 34px rgba(16, 49, 65, 0.18);
  color: var(--text-sub);
  font-size: 11.5px;
  line-height: 1.35;
  backdrop-filter: blur(16px) saturate(1.2);
  -webkit-backdrop-filter: blur(16px) saturate(1.2);
}
.offline-info strong {
  color: var(--text-main);
  font-size: 13px;
}
.offline-info > button {
  position: absolute;
  top: 5px;
  right: 5px;
  width: 28px;
  height: 28px;
  border: 0;
  border-radius: 8px;
  background: transparent;
  color: var(--text-dim);
  font-size: 18px;
```

```
  cursor: pointer;



}
.map-loading {
  position: absolute;
  inset: 0;
  z-index: 1200;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 9px;
  background: linear-gradient(145deg, #dfeaf1, #d1e0e9);
  color: #53636d;
  font-size: 12px;
}
.map-loading__spinner {
  width: 16px;
  height: 16px;
  border: 2px solid rgba(83, 99, 109, 0.2);
  border-top-color: #007aff;
  border-radius: 50%;
  animation: map-spin 0.8s linear infinite;
}
@keyframes map-spin {
  to { transform: rotate(360deg); }
}
.map-legend {
  display: flex;
  flex-wrap: wrap;
  justify-content: flex-end;
  gap: 10px;
  color: var(--text-sub);
  font-size: 12px;
  font-weight: 400;
}
.map-legend__item {
  display: flex;
  align-items: center;
  white-space: nowrap;
}
/* ---------- 本次派单决策（图上方一行） ---------- */
.map-decision {
  display: flex;
  align-items: center;
  gap: 8px;
  flex-wrap: wrap;
  margin: 0 12px 8px;
  padding: 7px 11px;
  border-radius: 10px;
  border: 1px solid color-mix(in srgb, #ff9500 34%, transparent);
  background: color-mix(in srgb, #ff9500 9%, transparent);
```

```
  font-size: 12px;


  line-height: 1.5;
}
.map-decision__tag {
  flex: 0 0 auto;
  padding: 1px 8px;
  border-radius: 999px;
  background: #ff9500;
  color: #ffffff;
  font-size: 11px;
  font-weight: 600;
  white-space: nowrap;
}
.map-decision__text { color: var(--text-main); min-width: 0; }
.map-decision__count {
  margin-left: auto;
  color: var(--text-dim);
  font-size: 11.5px;
  white-space: nowrap;
}
.map-tools {
  position: absolute;
  z-index: 1000;
  top: 10px;
  right: 10px;
  display: flex;
  flex-direction: column;
  gap: 7px;
  padding: 9px 10px;
  border: 1px solid rgba(255, 255, 255, 0.82);
  border-radius: 13px;
  background: rgba(252, 253, 254, 0.85);
  box-shadow:
    0 1px 2px rgba(24, 44, 58, 0.07),
    0 8px 24px rgba(24, 44, 58, 0.13);
  color: #4f5c63;
  font-size: 12px;
  backdrop-filter: blur(18px) saturate(1.25);
  -webkit-backdrop-filter: blur(18px) saturate(1.25);
}
.map-tools__base {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 3px;
  padding: 3px;
  border-radius: 9px;
  background: rgba(118, 128, 138, 0.13);
}
.map-tools__base button {
  min-height: 28px;
```

```
  padding: 4px 9px;
 }
 .map-tools__base button {
  border: 0;
  border-radius: 7px;
  background: transparent;
  color: #5c6971;
  font-size: 11px;
  font-weight: 550;
  cursor: pointer;
 }
.map-tools__base button.is-active {
  background: #ffffff;
  box-shadow: 0 1px 4px rgba(24, 44, 58, 0.14);
  color: #007aff;
 }
.map-tools__item {
  display: flex;
  align-items: center;
  gap: 6px;
  cursor: pointer;
  user-select: none;
 }
.map-tools__item input {
  width: 16px;
  height: 16px;
  margin: 0;
  accent-color: #007aff;
  cursor: pointer;
 }
:deep(.leaflet-container) {
  background: #d6e3eb;
  color: var(--text-main);
  font-family: inherit;
 }
:deep(.leaflet-tile-pane) {
  filter: saturate(1.03) contrast(1.04);
 }
:deep(.seasight-label-tiles) {
  filter: drop-shadow(0 1px 1px rgba(0, 0, 0, 0.35));
 }
:deep(.leaflet-control-zoom) {
  overflow: hidden;
  border: 1px solid rgba(255, 255, 255, 0.8) !important;
  border-radius: 13px !important;
  box-shadow:
    0 1px 2px rgba(24, 44, 58, 0.08),
    0 8px 24px rgba(24, 44, 58, 0.14) !important;
 }
:deep(.leaflet-control-zoom a) {
  width: 40px !important;
  height: 40px !important;
```

```
  border: 0 !important;




  background: rgba(255, 255, 255, 0.84) !important;
  color: #3f4a51 !important;
  font: 20px/40px -apple-system, BlinkMacSystemFont, sans-serif !important;
  backdrop-filter: blur(18px) saturate(1.25);
  -webkit-backdrop-filter: blur(18px) saturate(1.25);
}
:deep(.leaflet-control-zoom a + a) {
  border-top: 1px solid rgba(60, 60, 67, 0.13) !important;
}
:deep(.leaflet-control-zoom a:hover) {
  background: rgba(255, 255, 255, 0.98) !important;
  color: #007aff !important;
}
:deep(.leaflet-control-attribution) {
  padding: 3px 6px;
  border-radius: 7px 0 0 0;
  background: rgba(255, 255, 255, 0.72);
  color: #63717a;
  font-size: 10px;
  backdrop-filter: blur(10px);
  -webkit-backdrop-filter: blur(10px);
}
:deep(.leaflet-control-attribution a) {
  color: #466879;
}
:deep(.seasight-marker) {
  border: 0;
  background: transparent;
}
:deep(.seasight-marker svg) {
  display: block;
  width: 100%;
  height: 100%;
  overflow: visible;
  filter: drop-shadow(0 3px 5px rgba(6, 24, 35, 0.28));
}
:deep(.seasight-event-marker) {
  filter: drop-shadow(0 2px 4px rgba(6, 24, 35, 0.3));
}
:deep(.seasight-leaflet-heat) {
  filter: blur(1.3px);
}
:deep(.seasight-popup-shell .leaflet-popup-content-wrapper) {
  border: 1px solid rgba(60, 60, 67, 0.12);
  border-radius: 13px;
  background: rgba(255, 255, 255, 0.96);
  box-shadow: 0 14px 38px rgba(16, 49, 65, 0.2);
  backdrop-filter: blur(16px);
  -webkit-backdrop-filter: blur(16px);
```

```
}

:deep(.seasight-popup-shell .leaflet-popup-content) {
  margin: 11px 13px;
}
:deep(.seasight-popup-shell .leaflet-popup-tip) {
  background: rgba(255, 255, 255, 0.96);
}
:deep(.seasight-popup) {
  display: flex;
  min-width: 160px;
  flex-direction: column;
  gap: 3px;
  color: #5f6b73;
  font-size: 11px;
  line-height: 1.45;
}
:deep(.seasight-popup strong) {
  margin-bottom: 2px;
  color: #26333b;
  font-size: 13px;
}
:deep(.seasight-popup b) {
  display: inline-block;
  min-width: 42px;
  color: #879198;
  font-weight: 500;
}
@media (max-width: 760px) {
  .map-panel .panel-title {
    align-items: flex-start;
    flex-direction: column;
    gap: 8px;
  }
  .map-legend {
    width: 100%;
    justify-content: flex-start;
    flex-wrap: nowrap;
    gap: 12px;
    padding-bottom: 2px;
    overflow-x: auto;
    scrollbar-width: none;
  }
  .map-legend::-webkit-scrollbar {
    display: none;
  }
  .map-panel__body,
  #seasight-map {
    min-height: 300px;
  }
  .map-tools {
```

```
    top: 8px;
    right: 8px;
    display: grid;
    grid-template-columns: repeat(2, auto);
    gap: 7px 10px;
    padding: 7px 8px;
  }
  .map-tools__base {
    grid-column: 1 / -1;
  }
  .map-source-badge {
    bottom: 22px;
  }
  .offline-info {
    top: auto;
    right: 10px;
    bottom: 78px;
    left: 82px;
    min-width: 0;
    max-width: none;
  }
}
@media (max-width: 380px) {
  .map-panel__body,
  #seasight-map {
    min-height: 270px;
  }
  .map-tools {
    font-size: 11px;
  }
  .map-tools__item input {
    width: 15px;
    height: 15px;
  }
  .map-source-badge small {
    display: none;
  }
}
</style>
# ===== frontend/src/components/sandbox/PostFX.js =====
/**
 * 后期处理管线：HDR UnrealBloom + MSAA 前向合成
 *
 * 参考 three.js 官方示例 webgl_postprocessing_unreal_bloom 的管线结构：
 *   RenderPass → UnrealBloomPass → OutputPass
 * 其中 OutputPass 是必须的：renderer 渲染到 RenderTarget 时不做色调映射，
 * 由 OutputPass 在最后统一做 ACES + sRGB 转换，否则画面会发灰/过曝。
 *
 * 泛光在沙盘里的作用：让"发光语义"真正成立：
 *   - 阳光海面碎钻（>1 的镜面高光先晕开再压缩，才有波光粼粼）
```

```
 *   - 船体航行灯 / 事件光柱 / 目标浮标脉冲（辉光让远距目标一眼可辨）



 * 这是从"能看"到"高级"的关键一步。
 */
import * as THREE from 'three'
import { EffectComposer } from 'three/examples/jsm/postprocessing/EffectComposer.js'
import { RenderPass } from 'three/examples/jsm/postprocessing/RenderPass.js'
import { UnrealBloomPass } from 'three/examples/jsm/postprocessing/UnrealBloomPass.js'
import { OutputPass } from 'three/examples/jsm/postprocessing/OutputPass.js'
export class PostFX {
  /**
   * @param {THREE.WebGLRenderer} renderer
   * @param {THREE.Scene} scene
   * @param {THREE.Camera} camera
   */
  constructor(renderer, scene, camera) {
    const size = renderer.getDrawingBufferSize(new THREE.Vector2())
    // HalfFloat RT 承载 HDR 亮度（泛光阈值以上的值不能被 8bit 截断）；
    // samples:4 = WebGL2 MSAA：composer 会绕过 canvas 的 antialias 参数，必须显式开
    const rt = new THREE.WebGLRenderTarget(Math.max(2, size.x), Math.max(2, size.y), {
      type: THREE.HalfFloatType,
      samples: 4,
    })
    this.composer = new EffectComposer(renderer, rt)
    this.composer.addPass(new RenderPass(scene, camera))
    // (resolution, strength, radius, threshold)
    // 阈值 0.85：只让真正的高光参与泛光（阳光/碎钻/灯光），避免整幅画面发糊
    this.bloom = new UnrealBloomPass(
      new THREE.Vector2(Math.max(2, size.x), Math.max(2, size.y)),
      0.55,
      0.5,
      0.85,
    )
    this.composer.addPass(this.bloom)
    this.composer.addPass(new OutputPass())
  }
  setSize(width, height) {
    this.composer.setSize(width, height)
  }
  setPixelRatio(ratio) {
    this.composer.setPixelRatio(ratio)
  }
  render() {
    this.composer.render()
  }
  dispose() {
    for (const pass of this.composer.passes) pass.dispose?.()
    this.composer.renderTarget1?.dispose()
    this.composer.renderTarget2?.dispose()
  }
}
```